Database Internals: Query Optimization

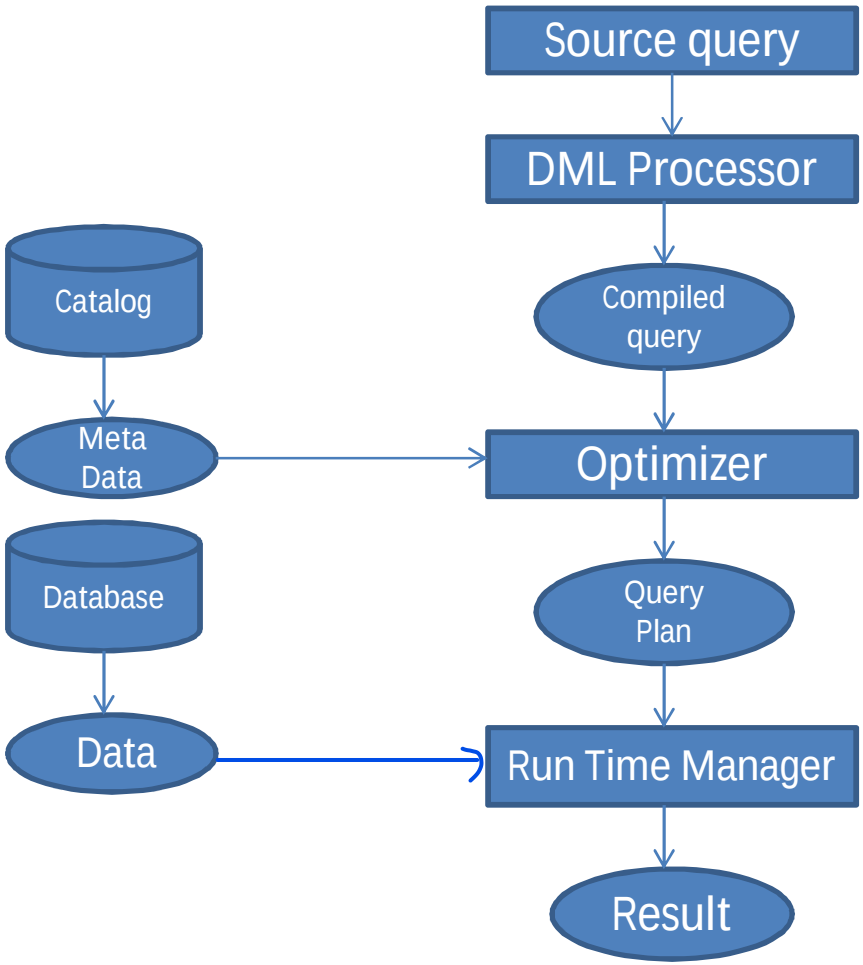
Sankhayan Choudhury
Professor, Dept. of Computer Science & Engineering
University of Calcutta

Goal: Query Processing

- To transform a query written in a high level language (SQL) into a correct and efficient execution strategy expressed in a low level language(Relational Algebra)

- To execute the strategy for retrieving the required data

Query Processing: Flowchart



Query decomposition

R.A. Expression

Expression transformation, cost estimation etc

Optimized code

Execution

Optimization?

- Why Optimization?

  Many equivalent Relational Algebra expressions may exist for a same high level query written in SQL.

  The aim to choose the best one that minimizes resource usage.

Optimizing Criteria?

- Resource Usage directly maps to Response time
- Optimization is done based on no of secondary memory access.

Illustration

- Select * from staff s, branch b

  where s.branch No=b.branchNo
  And(s.position=‘Manager’ AND
  b.city=‘London’);

Illustrative example

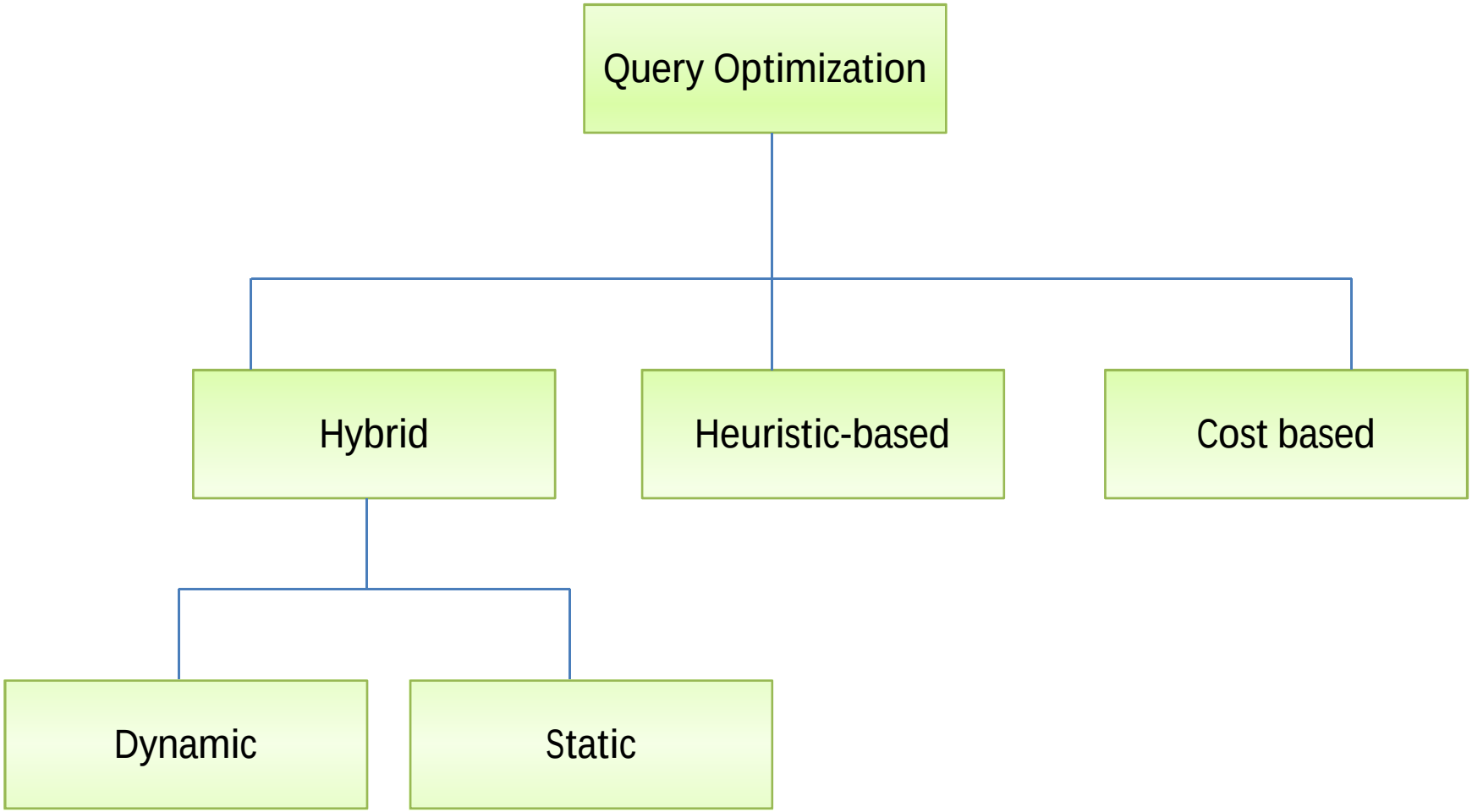
- Assumptions: No. of tuples in staff=1000, No of tuples in Branch=50,
- 50 managers & 5 London branches are there

Opt1: (1000+50)+ 2*(1000*50)= 101050

Opt2: 2*1000+(1000+50)= 3050
1000+2*50+50+2*5

Opt3:1000+(2*50)+5+(50+5)=1160
(1000+(50+5)+(50+5)+50)

Classification



Heuristic Based Vs. Cost Based

- Cost the query into internal form
- Convert to canonical form

- Choose candidate low-level procedures
- Generate query plans and choose the cheapest.

Heuristic Based Optimization

- **General thumb rule: Reduce tables as much as possible before performing binary operations.**

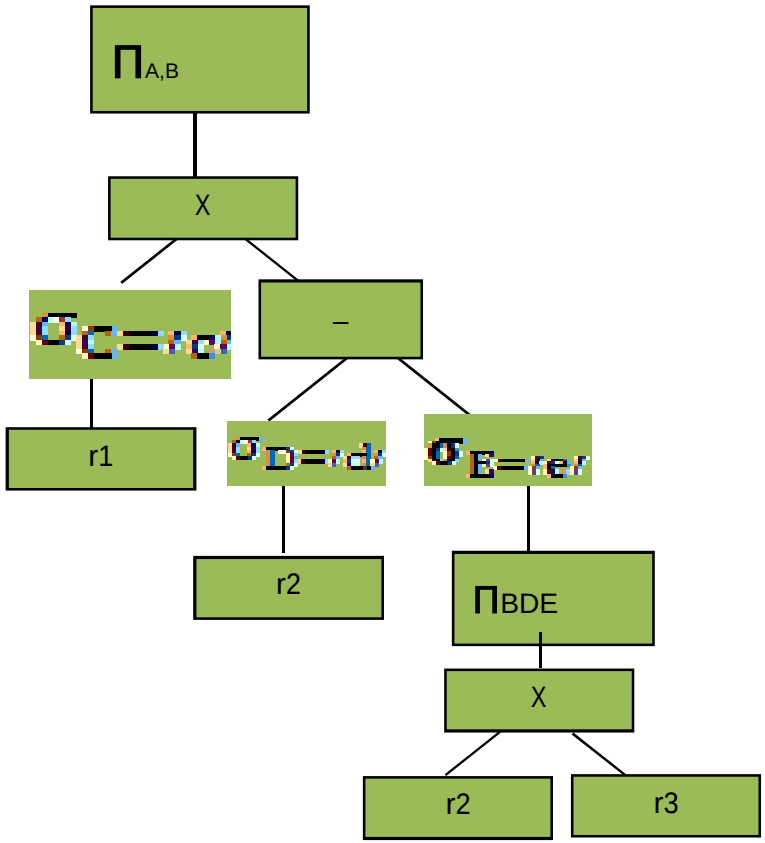
Strategies

- Perform selection operations as early as possible
- Perform projection operations as early as possible
- Compute common expression once
- Combine the Cartesian product with a subsequent selection operation whose predicate represents a join conditions into a join condition.

Query Tree

$$\pi_{A,B}(\sigma_{C='c'}(r1) \bowtie (\sigma_{D='d'}(r2) - \sigma_{F='e'}\pi_{BDE}(r2 \bowtie r3)))$$

r1,r2 and r3 are relations over R1(A,B,C), R2(B,D,E), R3(D,E)

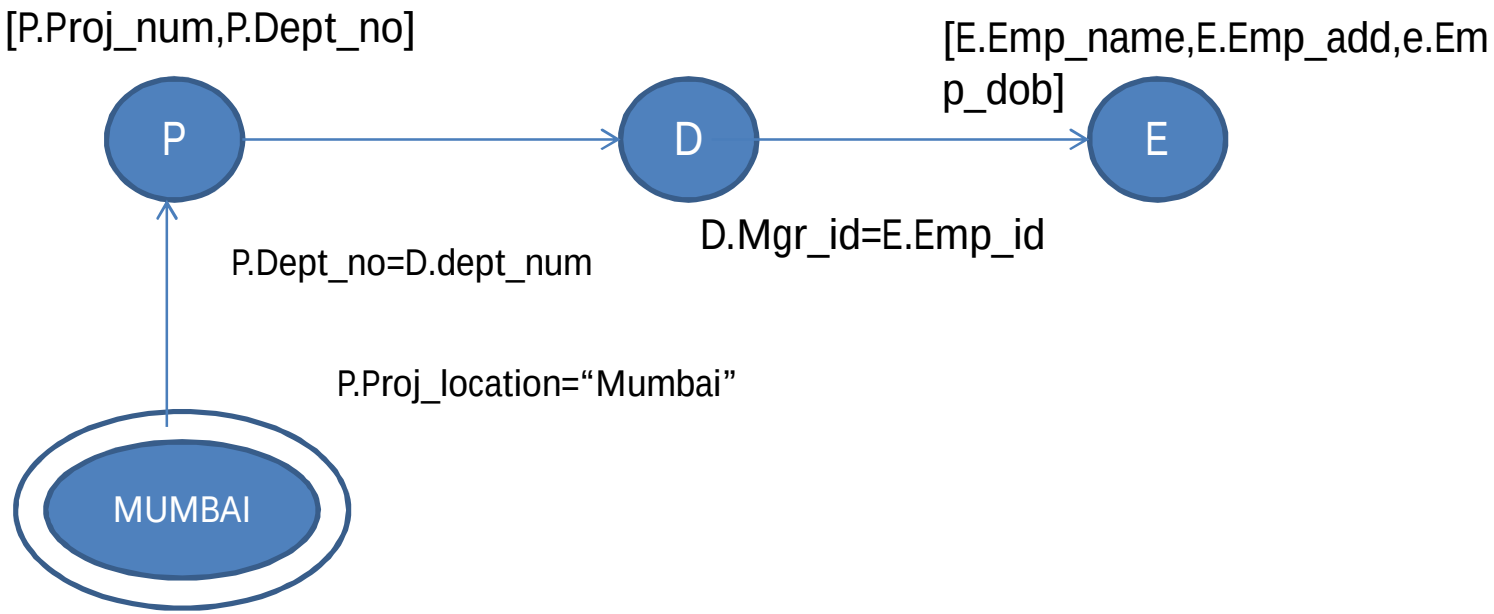


Query graph notation may be used. Disadvantage: It does not indicate an order of execution of the operators.

Query Grapgh

- A query graph representation corresponds to a relational calculus expression. In query graph,
  - Relation nodes are displayed as single circle
  - The const values from the query selection are represented by constant nodes[displayed as double circle.]
  - The selection and join conditions are represented by the graph edges
  - The attributes to be retrieved from each relation are displayed in square brackets above each relation

- Select P.Proj_no,P.Dept_no,E.name, E.Add, E.dob from project as P, Department as D, Employee as E where P.dept_no=D.D_num AND D_mgr_id=E.Emp_id AND P.proj_location='Mumbai'
- Query graph does not indicate an order on which operation to perform first like a query tree. Thus query tree representation is preferred one.



Illustrative Examples

SELECT LnameFROM EMPLOYEE,*WORKS_ON, PROJECT*
WHERE Pname='Aquarius' AND Pnumber=Pno AND
Essn=Ssn AND Bdate > '1957-12-31';

- Steps in converting a query tree during heuristic optimization.

(a) Initial (canonical) query tree for SQL query Q.

(b) Moving SELECT operations down the query tree.

(c) Applying the more restrictive SELECT operation first.

(d) Replacing CARTESIAN PRODUCT and SELECT with JOIN operations.

(e) Moving PROJECT operations down the query tree.

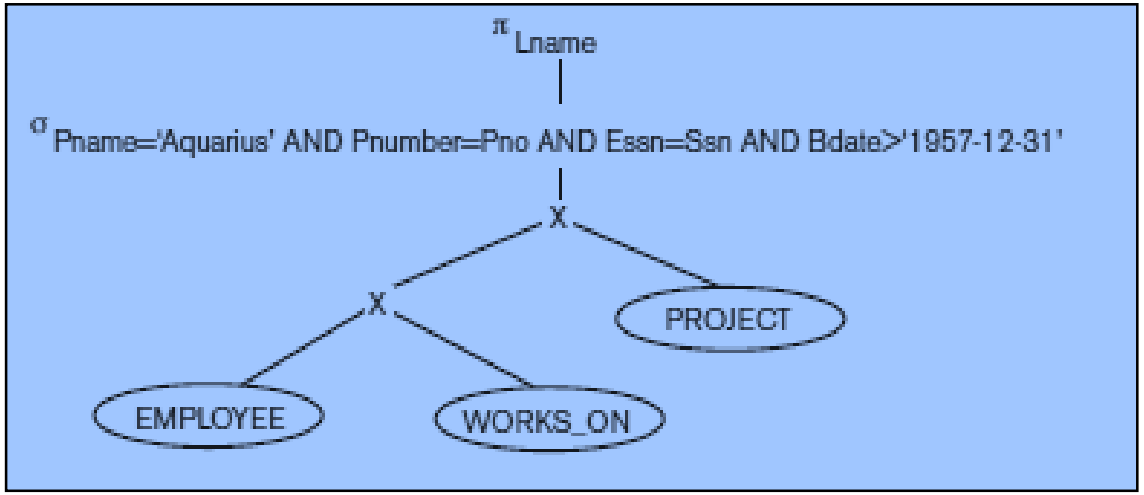

π Lname
σ Pname='Aquarius' AND Pnumber=Pno AND Essn=Ssn AND Bdate>'1957-12-31'
X
X
PROJECT
EMPLOYEE
WORKS_ON

Fig a

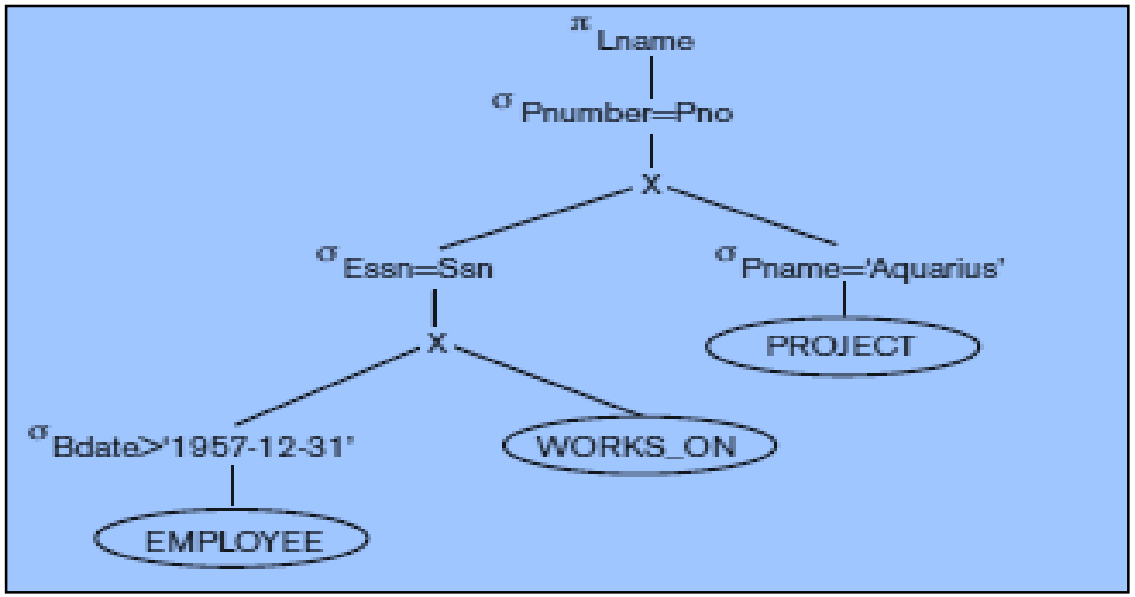

π Lname
σ Pnumber=Pno
X
σ Essn=Ssn
σ Pname='Aquarius'
X
PROJECT
σ Bdate>'1957-12-31'
WORKS_ON
EMPLOYEE

Fig b

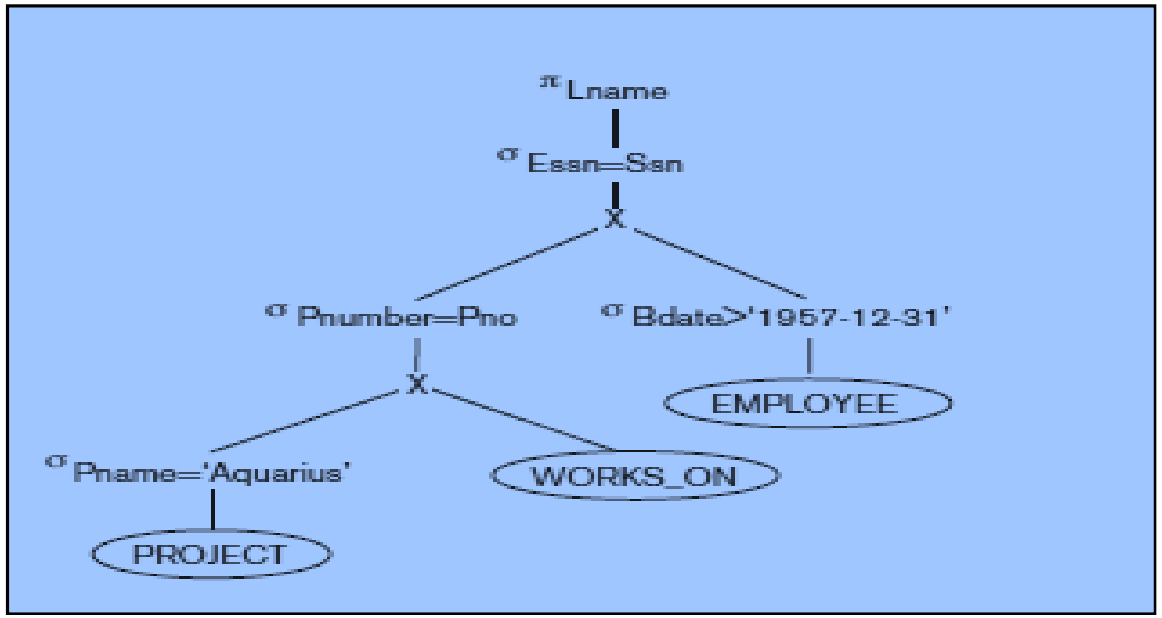

π Lname
σ Essn=Ssn
X
σ Pnumber=Pno
σ Bdate>'1957-12-31'
X
EMPLOYEE
σ Pname='Aquarius'
WORKS_ON
PROJECT

Fig c

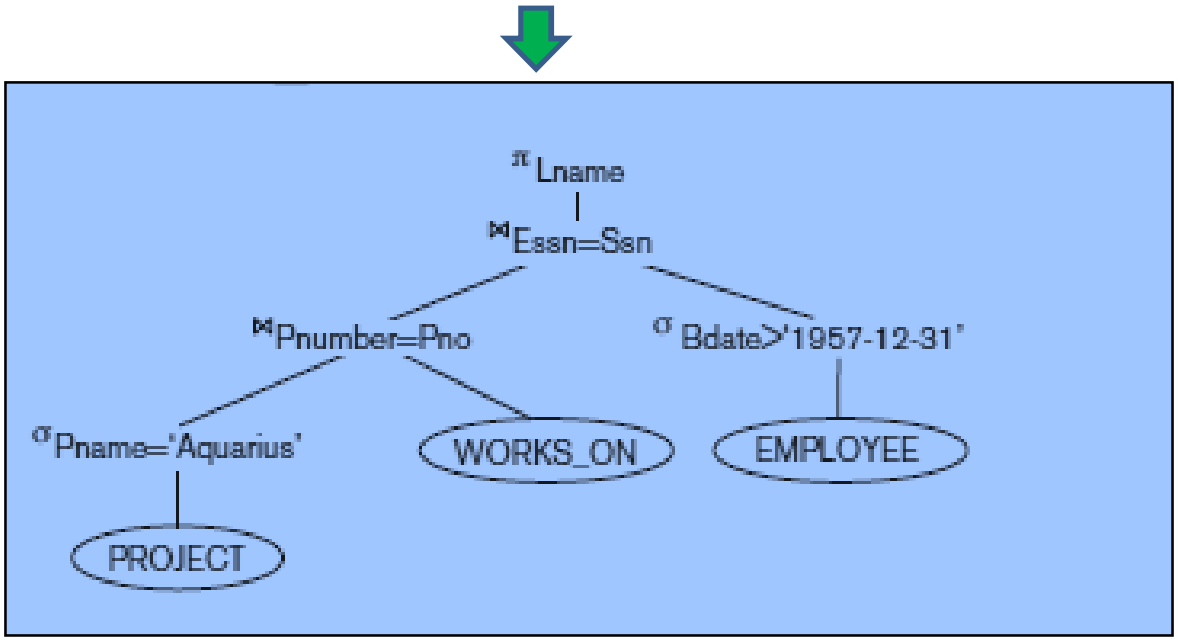

π Lname
⋈ Essn=Ssn
⋈ Pnumber=Pno
σ Bdate>'1957-12-31'
σ Pname='Aquarius'
WORKS_ON
EMPLOYEE
PROJECT

Fig d

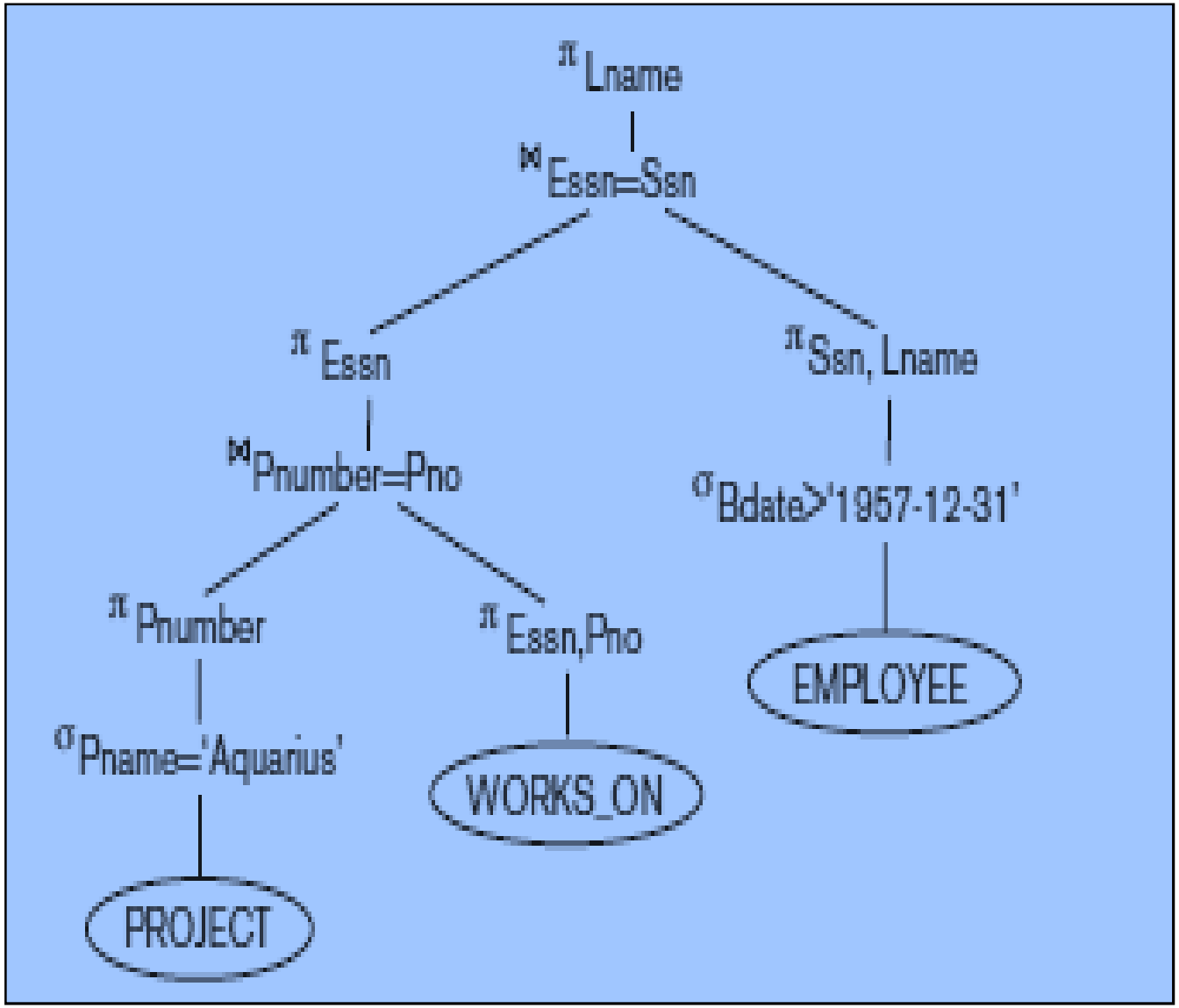

π Lname
⋈ Essn=Ssn
π Essn
π Ssn, Lname
⋈ Pnumber=Pno
σ Bdate>'1957-12-31'
π Pnumber
π Essn,Pno
EMPLOYEE
σ Pname='Aquarius'
WORKS_ON
PROJECT

Fig e

Cost Based Optimization

- Cost of an operation (not an expression) depends on low level access paths e.g.
- Index Schemes & Structure
- Algorithms

A typical Join Query

We will focus on SPJ, or Select-Project-Join-Queries

Select <attribute list>

From <relation list>

Where <condition list>

Example Join Query over R(A,B,C) and S(C,D,E):

Select B, D

From R, S

Where R.A = “c” $\wedge$ S.E = 2 $\wedge$ R.C = S.C

R

A	B	C
a	1	10
b	1	20
c	2	10
d	2	35
e	3	45

S

C	D	E
10	x	2
20	y	2
30	z	2
40	x	1
50	y	3

Select B,D

From R,S

Where R.A = “c” ∧ S.E

= 2 ∧ R.C=S.C

Answer

B	D
2	x

- How do we execute this query?

Select B,D

From R,S

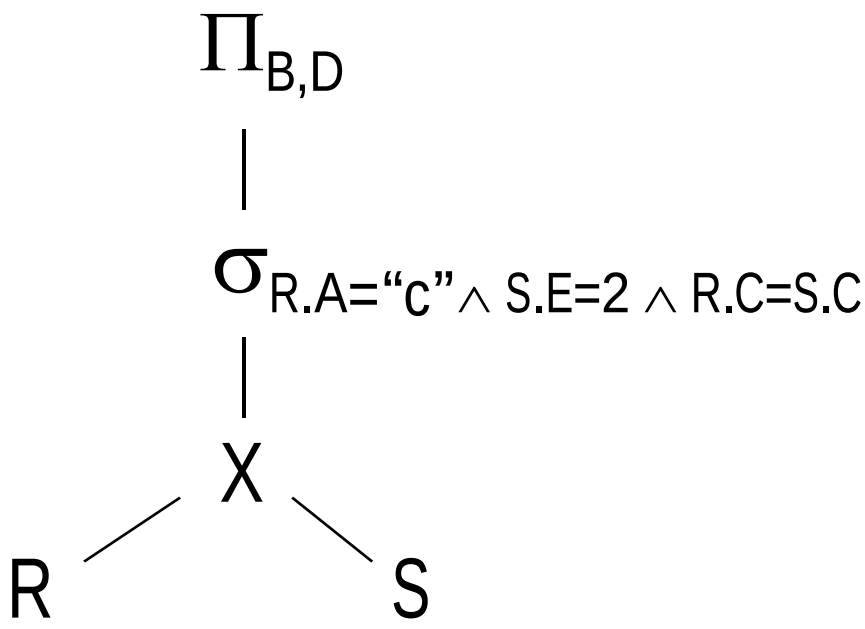
Where R.A = “c” $\wedge$ S.E = 2 $\wedge$ R.C=S.C

One idea

- Do Cartesian product
- Select tuples
- Do projection

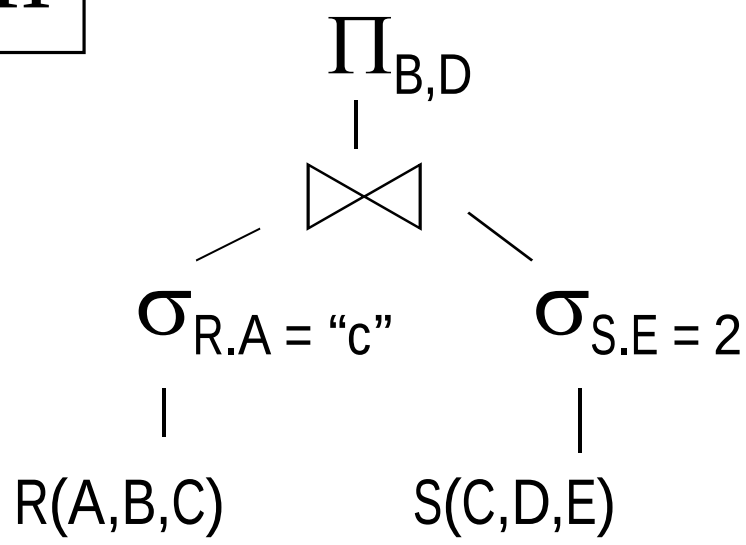
Relational Algebra - can be used to describe plans

Ex: Plan I



Another idea:

Plan II



$\bowtie$ natural join

Select B,D

From R,S

Where R.A = “c” $\wedge$ S.E

= 2 $\wedge$ R.C=S.C

R

A	B	C
a	1	10
b	1	20
c	2	10
d	2	35
e	3	45

σ (R)

A	B	C
c	2	10

σ(S)

C	D	E
10	x	2
20	y	2
30	z	2

S

C	D	E
10	x	2
20	y	2
30	z	2
40	x	1
50	y	3

$\bowtie$

Select B,D

From R,S

Where R.A = “c” $\wedge$ S.E

= 2 $\wedge$ R.C=S.C

Plan III

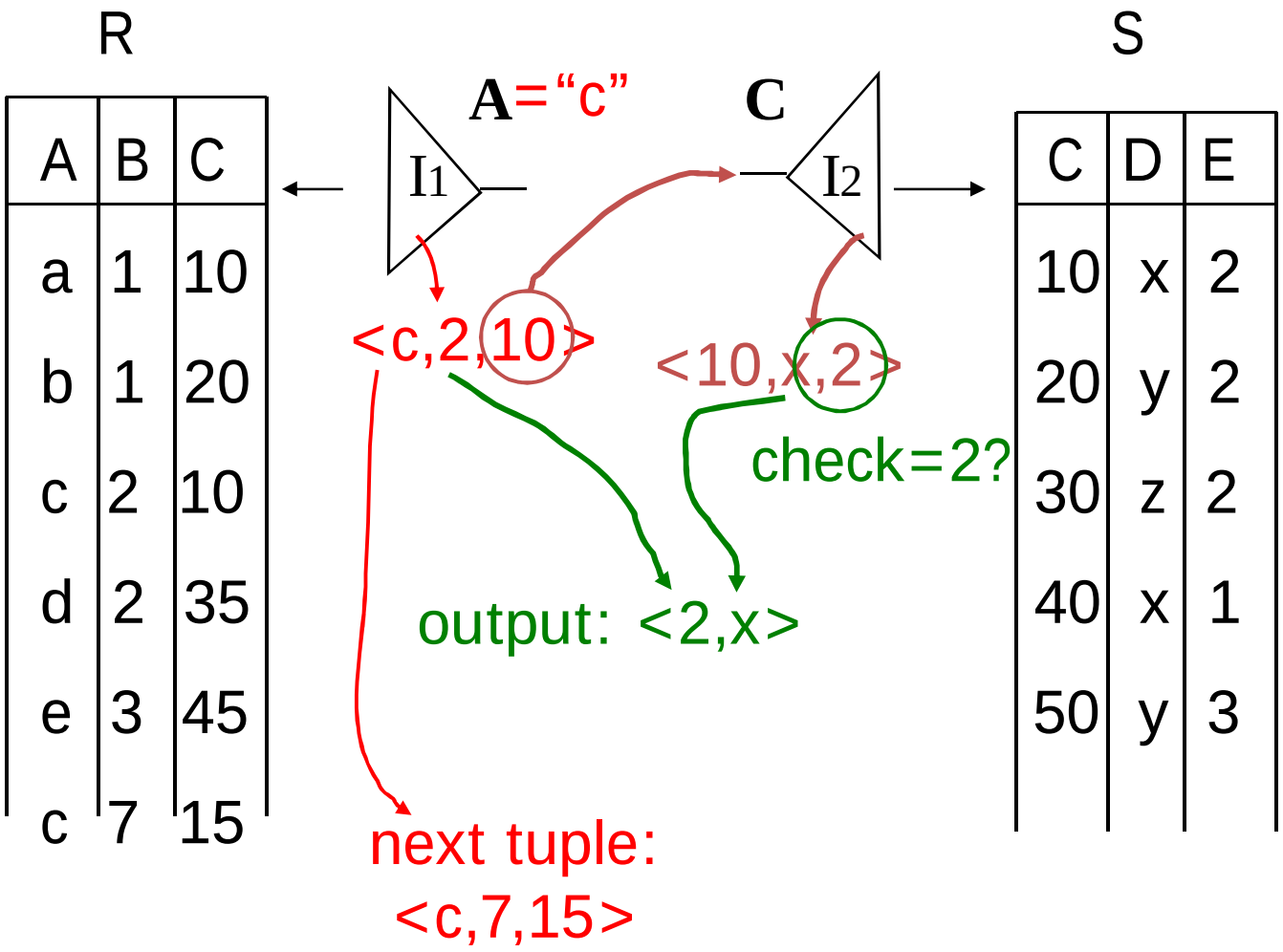
Use R.A and S.C Indexes

(1) Use R.A index to select R tuples with R.A = “c”

(2) For each R.C value found, use S.C index to find matching tuples

(3) Eliminate S tuples S.E $\neq$ 2

(4) Join matching R,S tuples, project B,D attributes, and place in result



A	B	C
a	1	10
b	1	20
c	2	10
d	2	35
e	3	45
c	7	15

C	D	E
10	x	2
20	y	2
30	z	2
40	x	1
50	y	3

Index Schemes

- Data structures that efficiently evaluate a class of filter predicates over a relation
- Class of filter predicates:
  - Single or multi-attributes (index-key attributes)
  - Range and/or equality predicates
- (Usually) independent of physical storage of relation:
  - Multiple indexes per relation

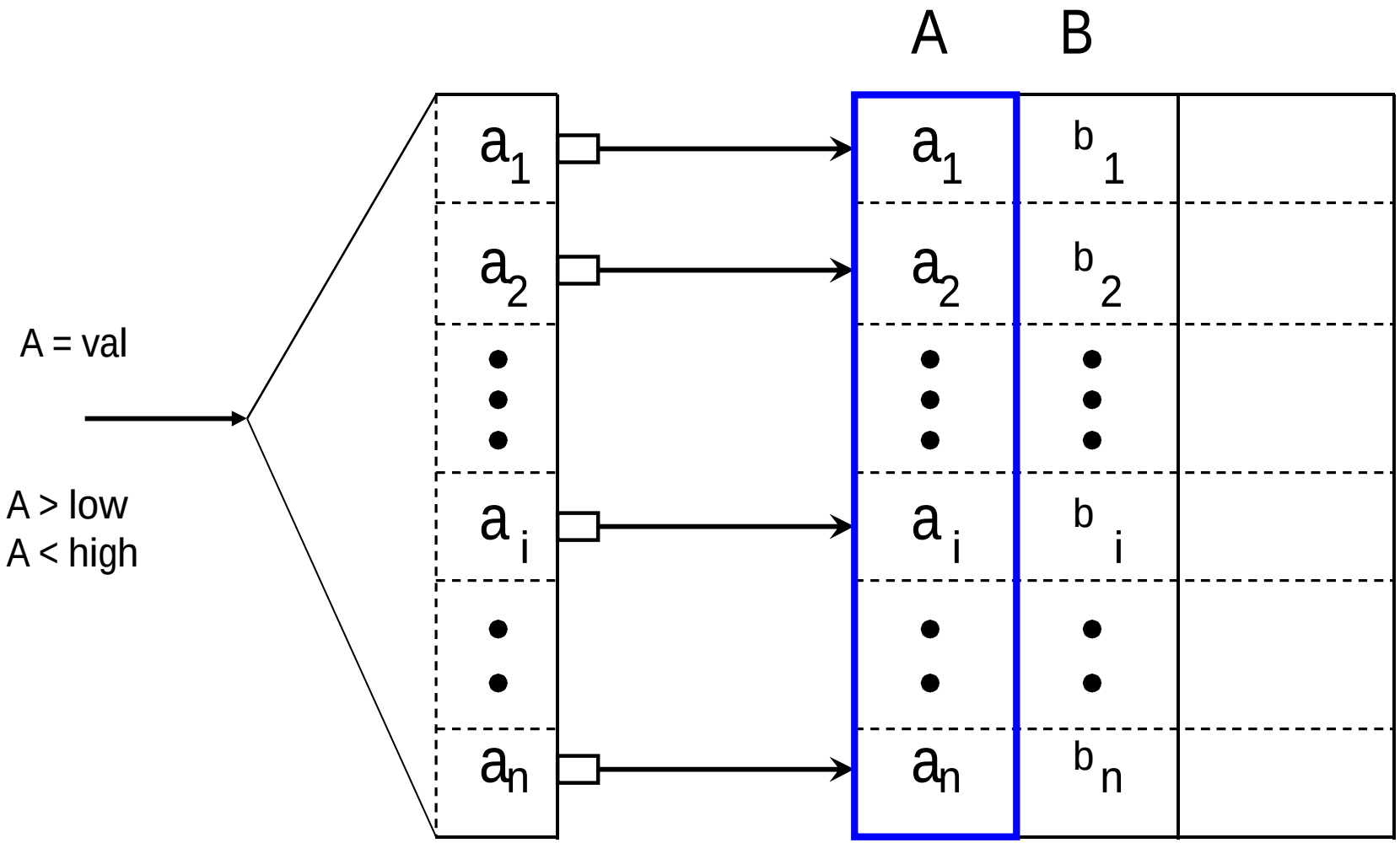
Cost Model

- B Data Pages with R Records / Page
- Average time to read/ write a disk Page = D
- Average time to process a record = C
- Time required to apply a Hash function to a record = H
- In general D = 15 milliseconds whereas
- C & H= 100 nano-seconds
- Thus the Cost of I/O will be the dominating Cost and the cost metric will be no. of disk page I/Os.

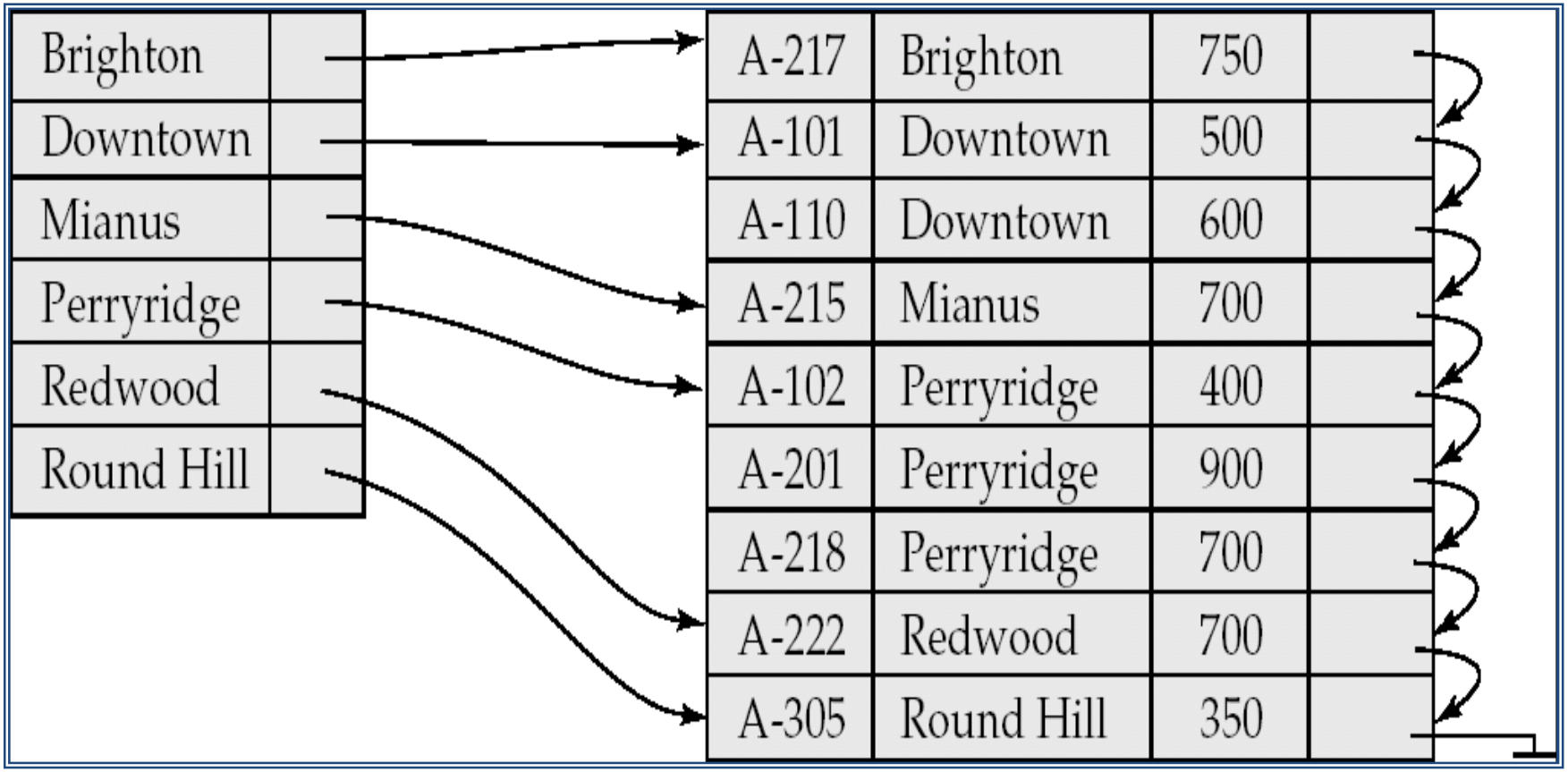
Index Schemes

- Index is an auxiliary structure to speed up operations that are not efficiently supported by file organization.
- Class of filter predicates:
  - Single or multi-attributes (index-key attributes)
  - Range and/or equality predicates
- (Usually) independent of physical storage of relation:
  - Multiple indexes per relation

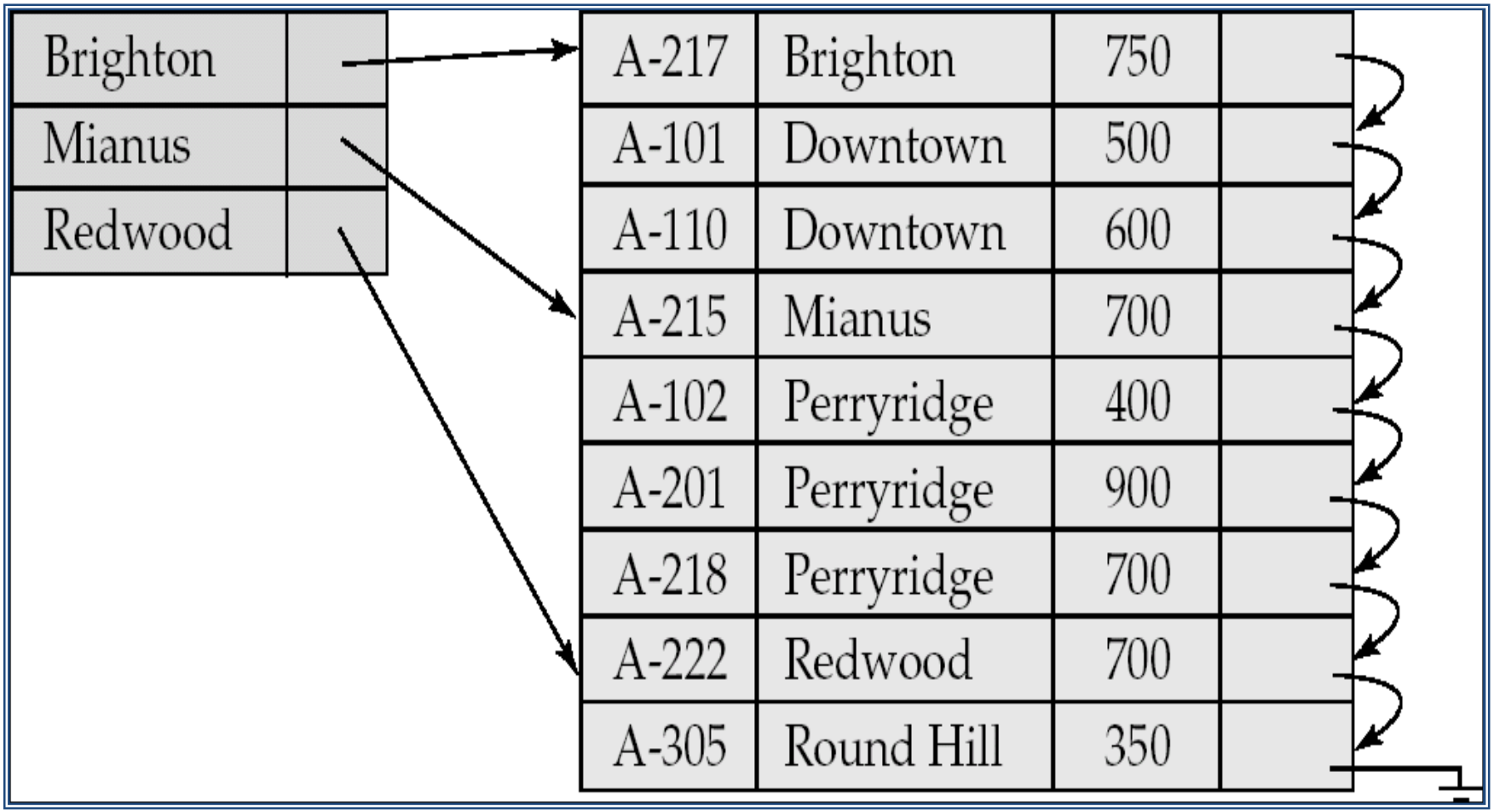
Single Attribute Index: General Construction



Dense Index


Brighton
Downtown
Mianus
Perryridge
Redwood
Round Hill
A-217 Brighton 750
A-101 Downtown 500
A-110 Downtown 600
A-215 Mianus 700
A-102 Perryridge 400
A-201 Perryridge 900
A-218 Perryridge 700
A-222 Redwood 700
A-305 Round Hill 350

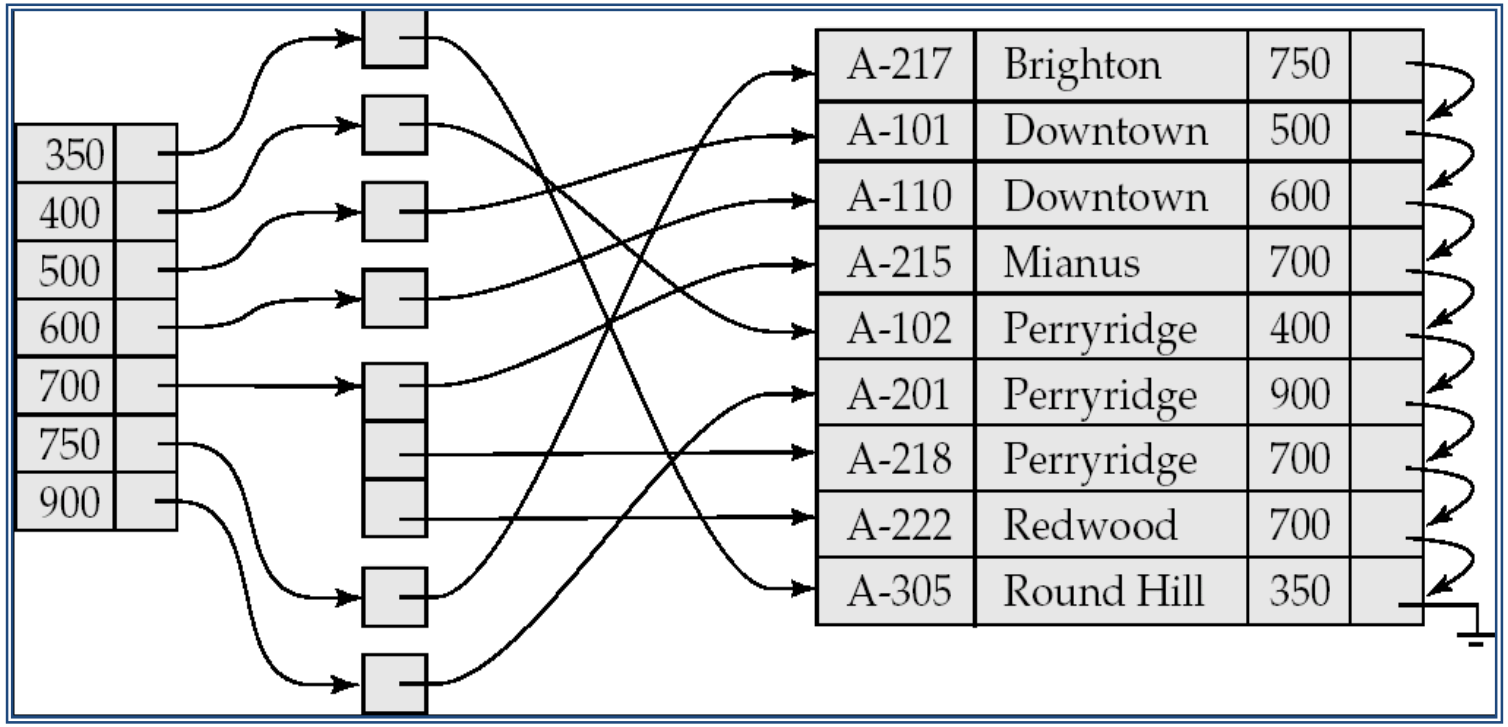
Sparse Index



Sparse Vs. Dense

- Compared to dense indices:
  - Less space and less maintenance overhead for insertions and deletions.
  - Generally slower than dense index for locating records.
- **Good tradeoff**: sparse index with an index entry for every block in file, corresponding to least search-key value in the block.

Secondary Index

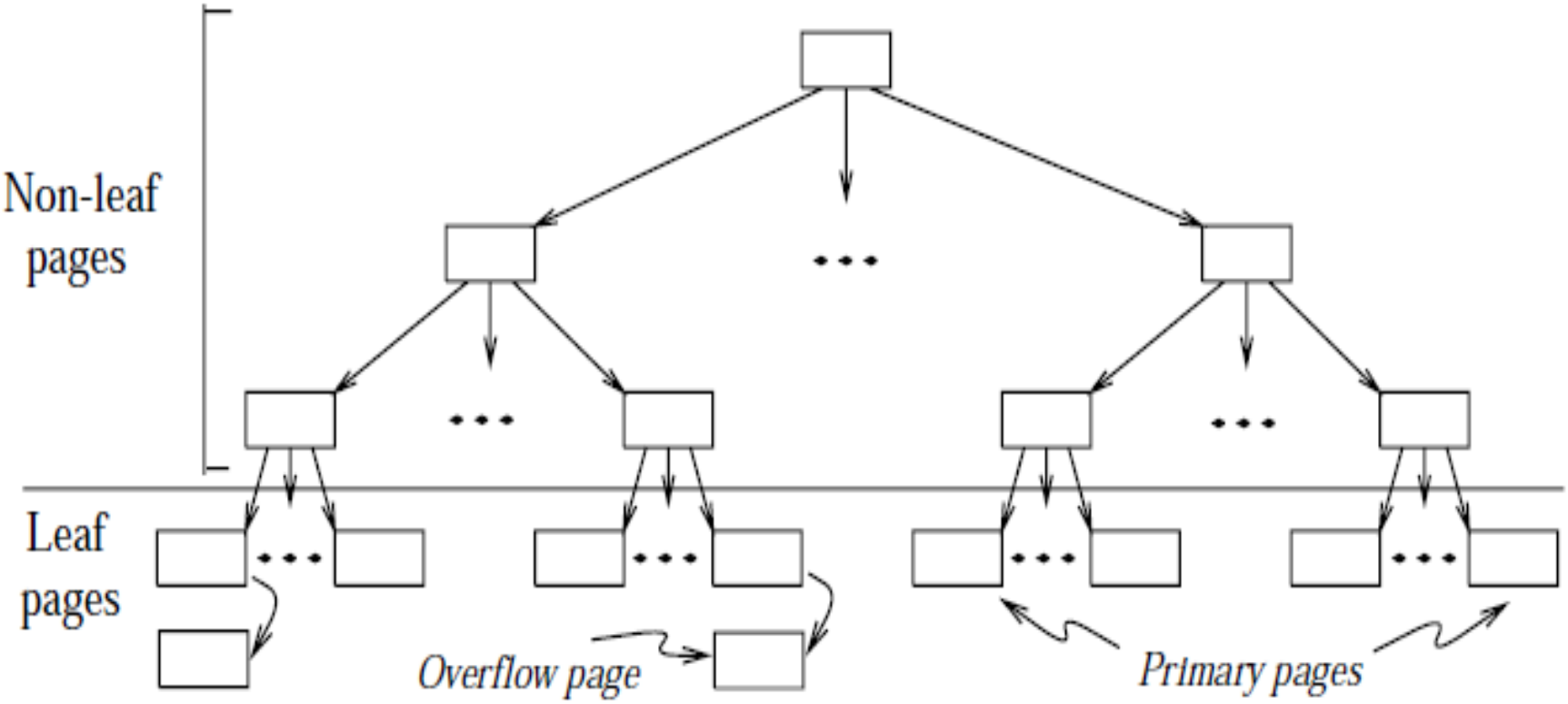


Provision in RDBMS

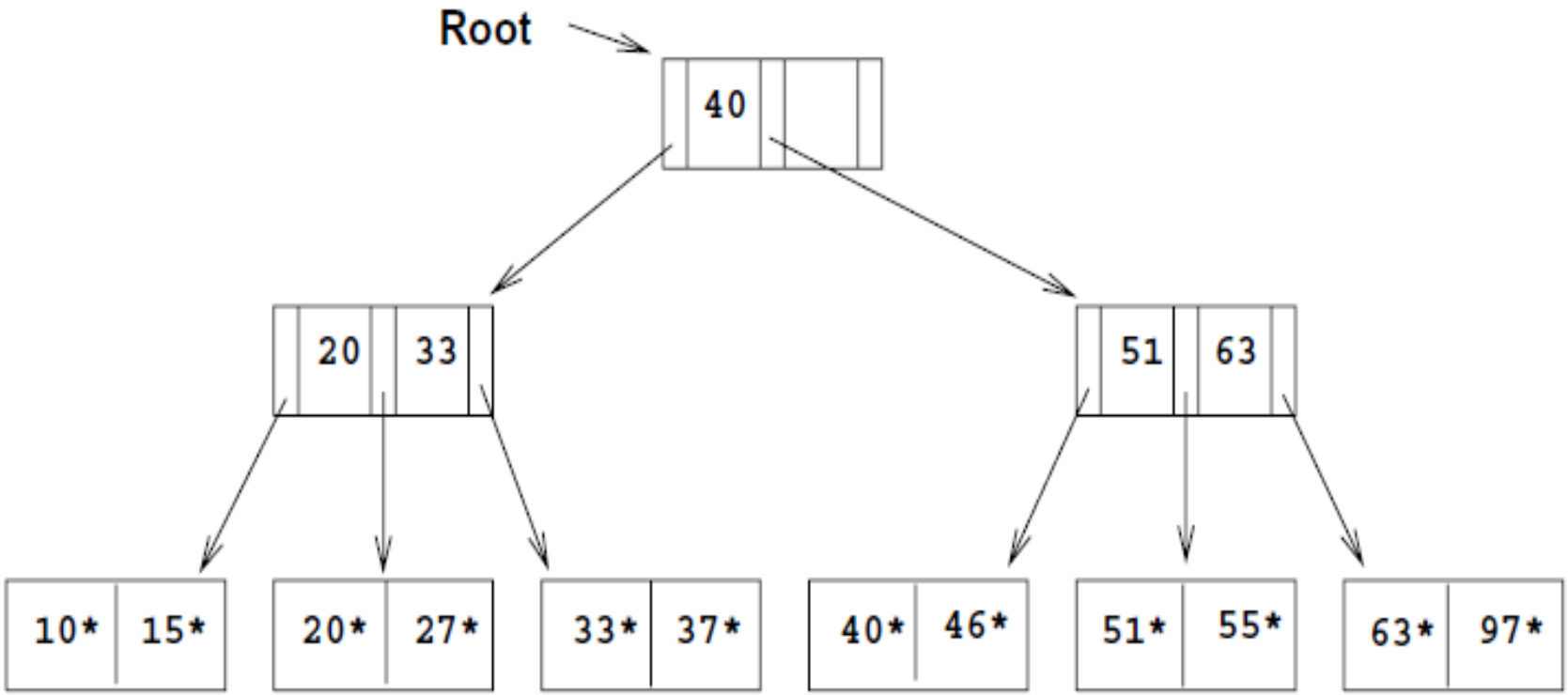
```
CREATE INDEX IndAgeRating ON Students
        WITH  STRUCTURE = BTREE,
              KEY = (age, gpa)
```

- Consider a file of students records sorted by gpa.
- **Query: Find all studnts with gpa higher than 3.0.**
- **To locate the first record do binary search and then scan from that point till condition holds.**
- **How could we improve the situation?**

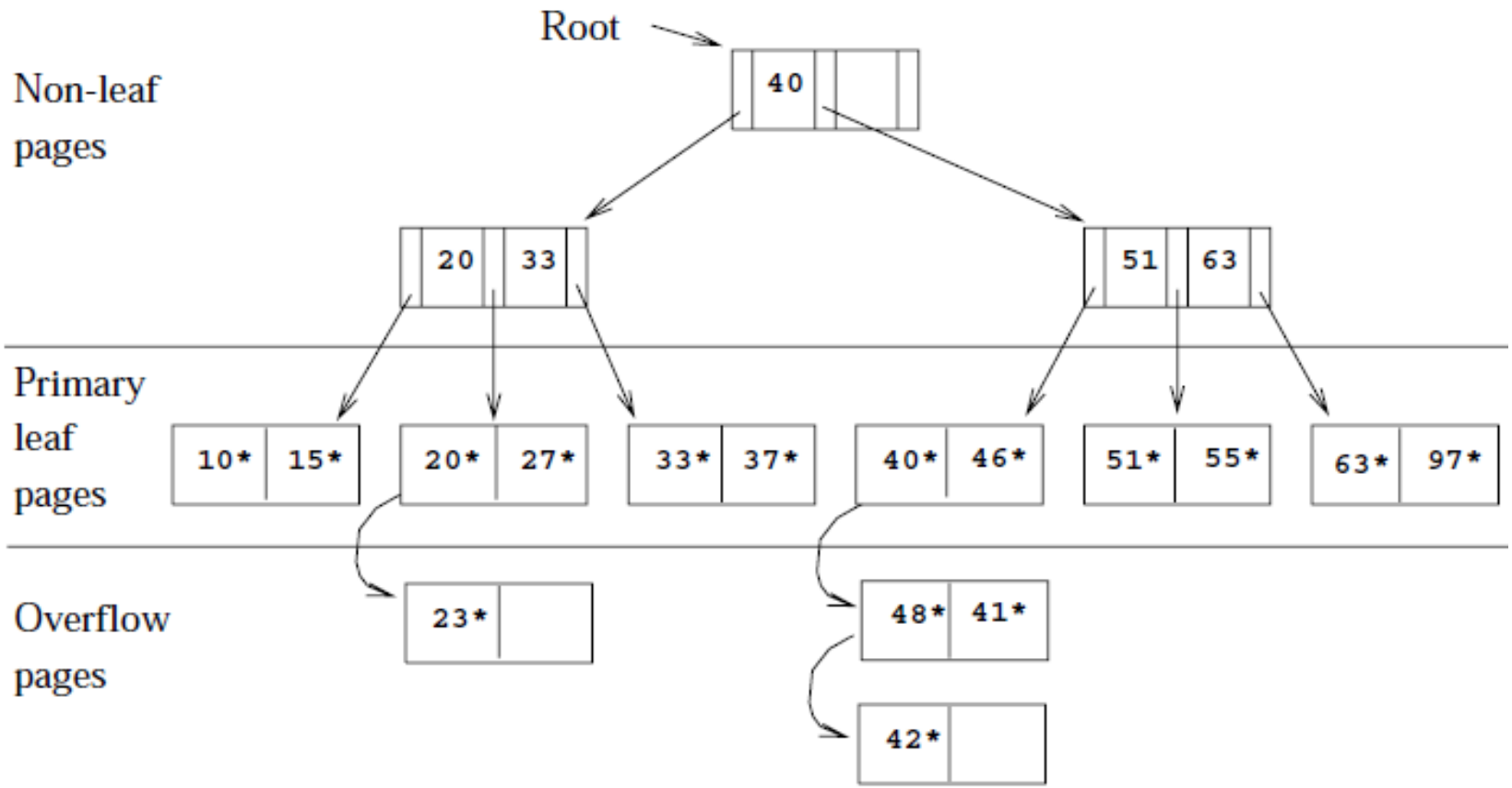
Tree based Indexing: ISAM



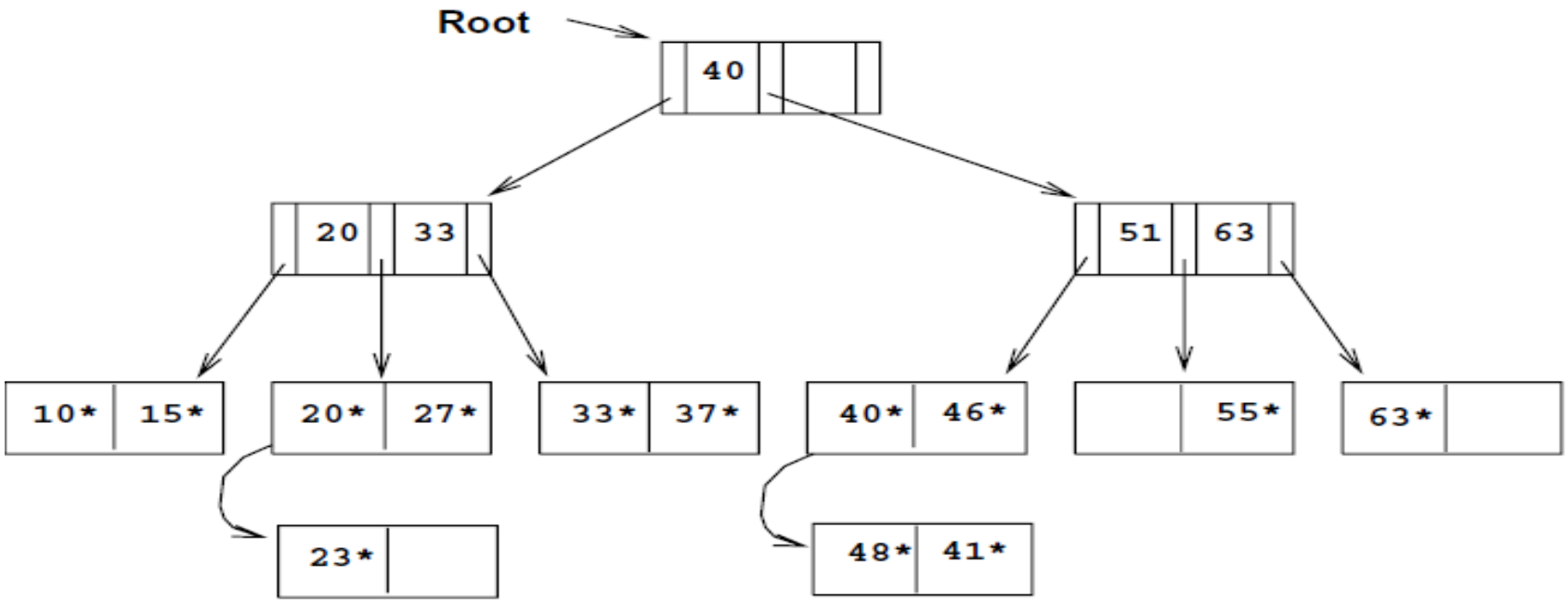
Sample ISAM tree



ISAM tree: After Insertion



Deletion of 42,51,97



Performance: ISAM

- No. of Disk I/O = No. of levels in the tree =

  $\log_F N$, where N = No. of Primary leaf pages & F, fanout = No. of Children/ Index Page

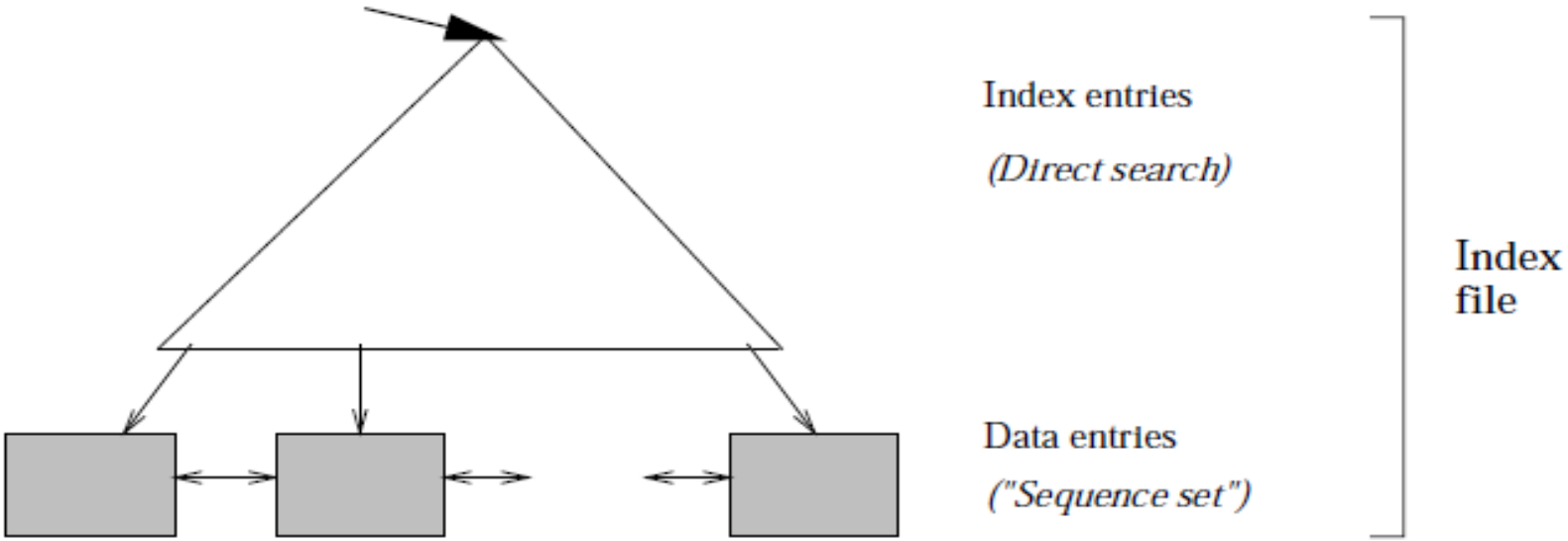
Example

File = 1,000,000 records, 10 records/ Leaf Page, 100 entries / Index Page

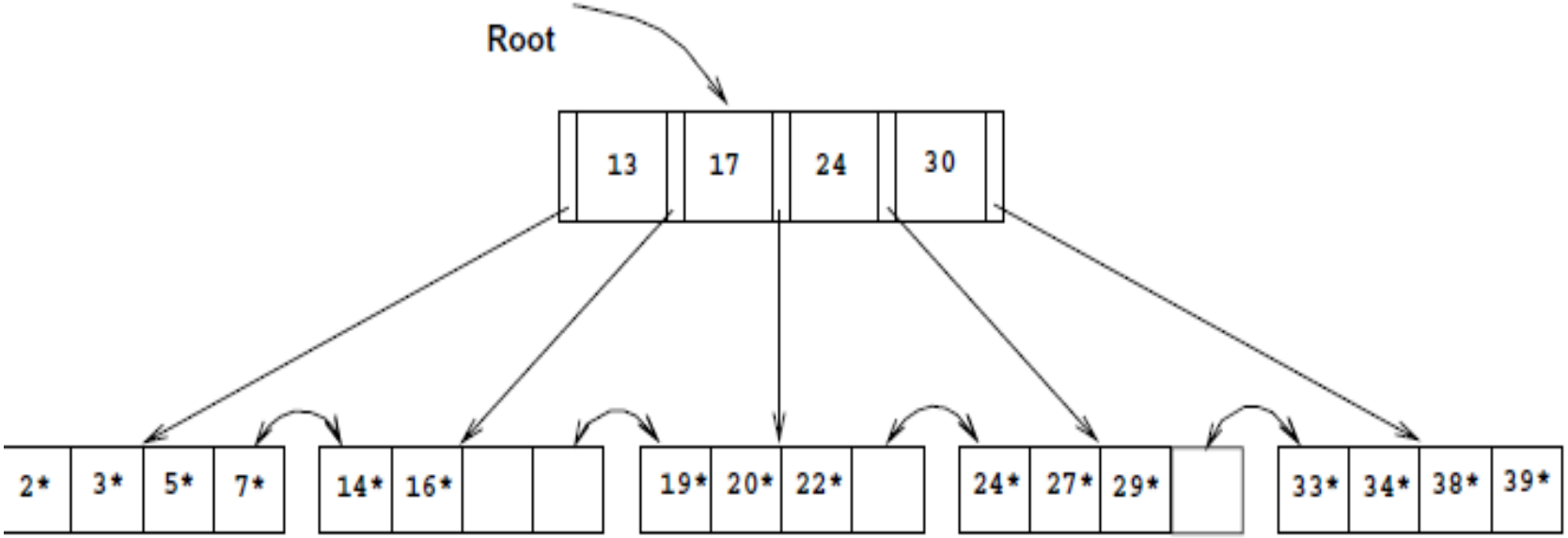
Cost in I/O of a

file scan= 1,00,000; Binary search of a sorted Data file = 17, Binary search with one level Index = 10 and ISAM = 3 (assuming no overflow)

Structure of a B+ tree



Example of a B+ tree of order = 2



Insertion of 8

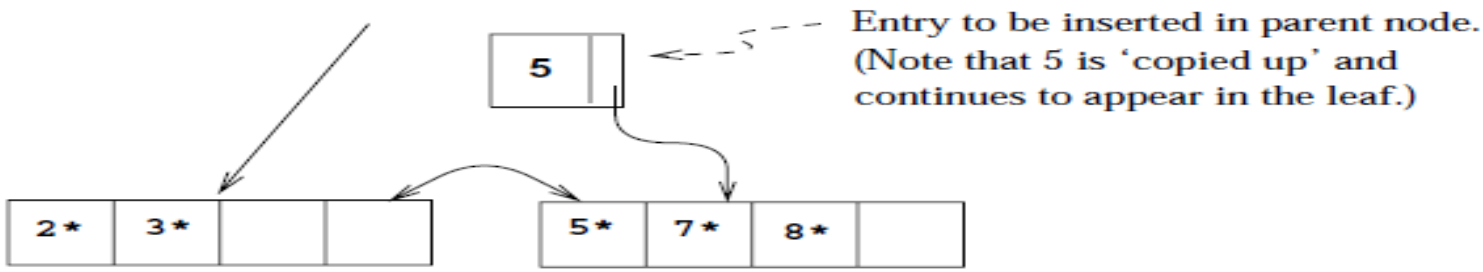
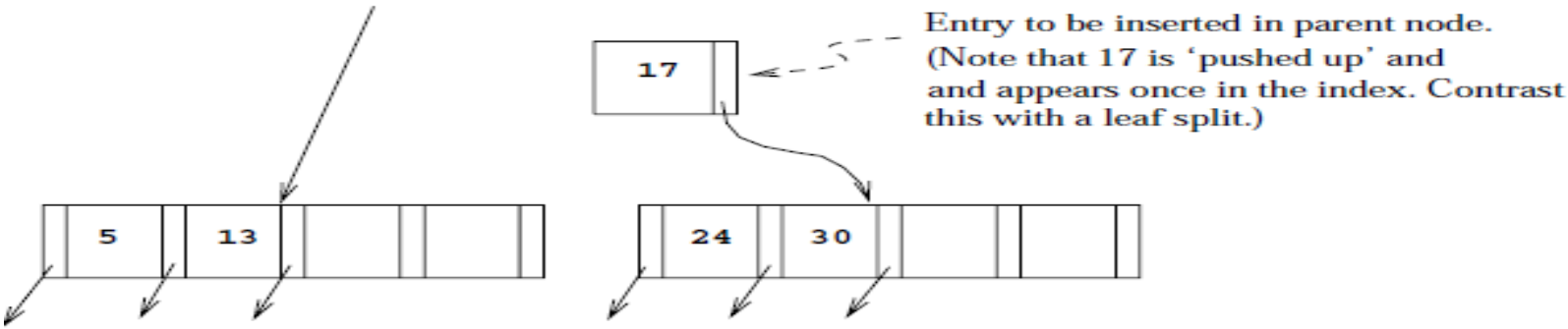


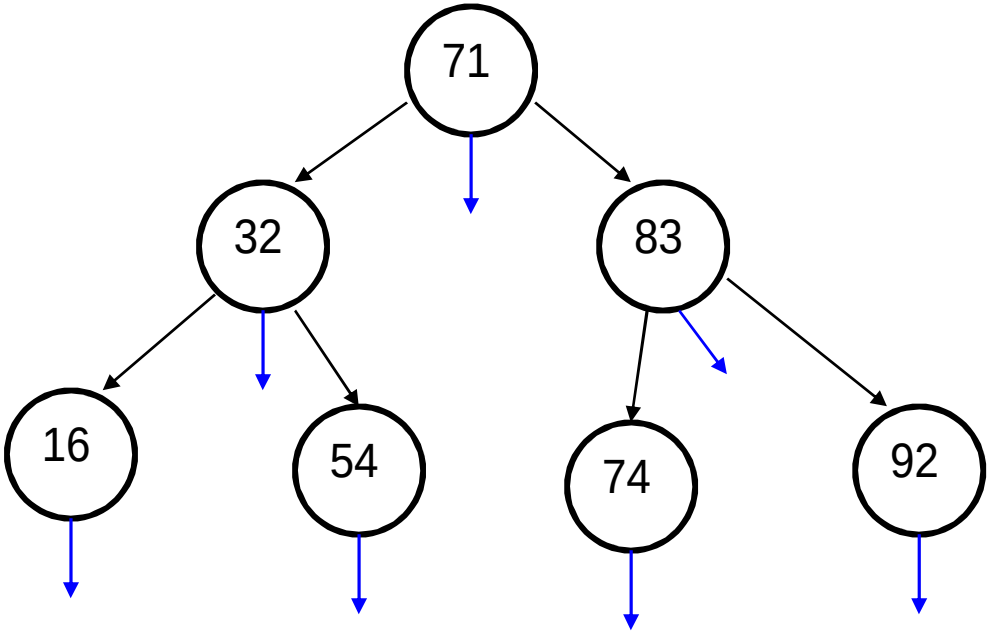
Figure 9.12 Split Leaf Pages during Insert of Entry 8*

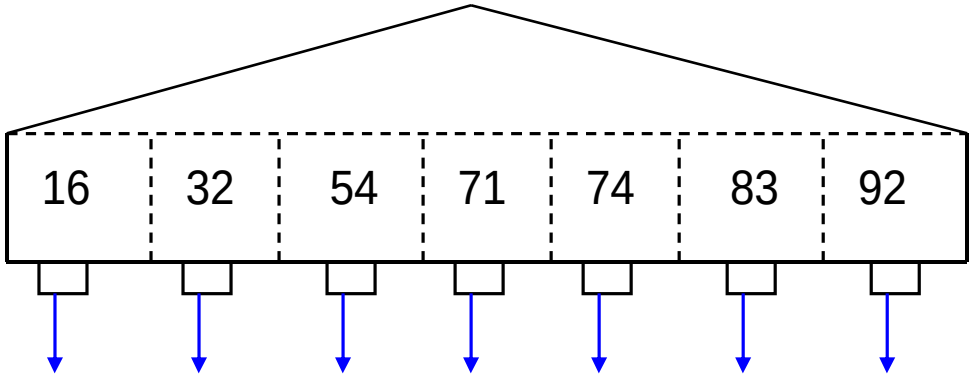


Order based Index: B-Trees

- Adaptation of search tree data structure
- Supports range predicates (and equality)

Use Binary Search Tree Directly?





Use Binary Search Tree Directly?

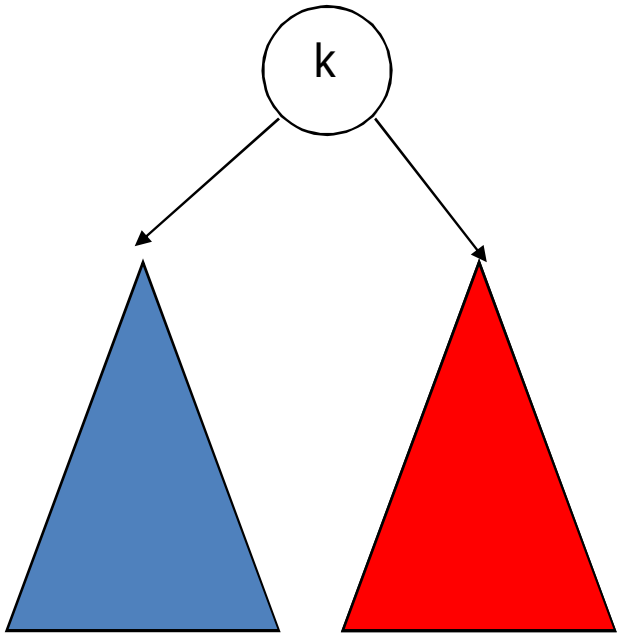
- Store records of type
  <key, left-ptr, right-ptr, data-ptr>
- Remember position of root
- Question: will this work?
  - Yes
  - But we can do better!

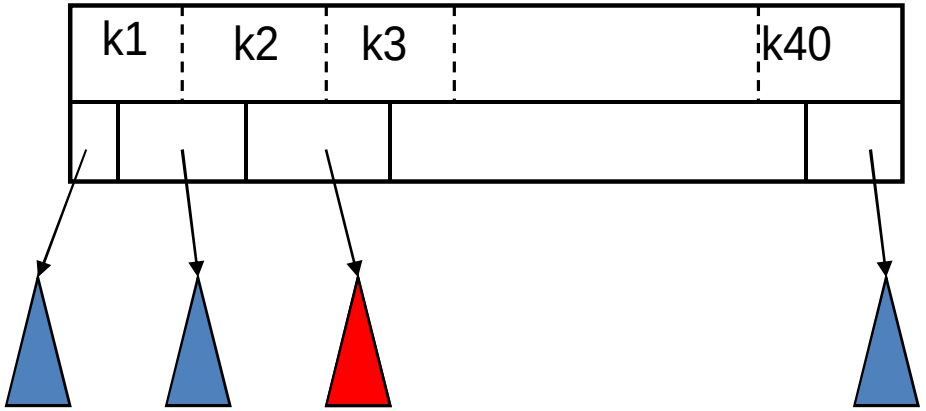
Use Binary Search Tree Directly?

- Number of keys: 1 M
- Number of levels: log (2^20) = 20
- Total cost index lookup: 20 random disk I/O
  - 20 x 20 ms = 400 ms

B-Tree: less than 3 random disk I/O

B-Tree vs. Binary Search Tree

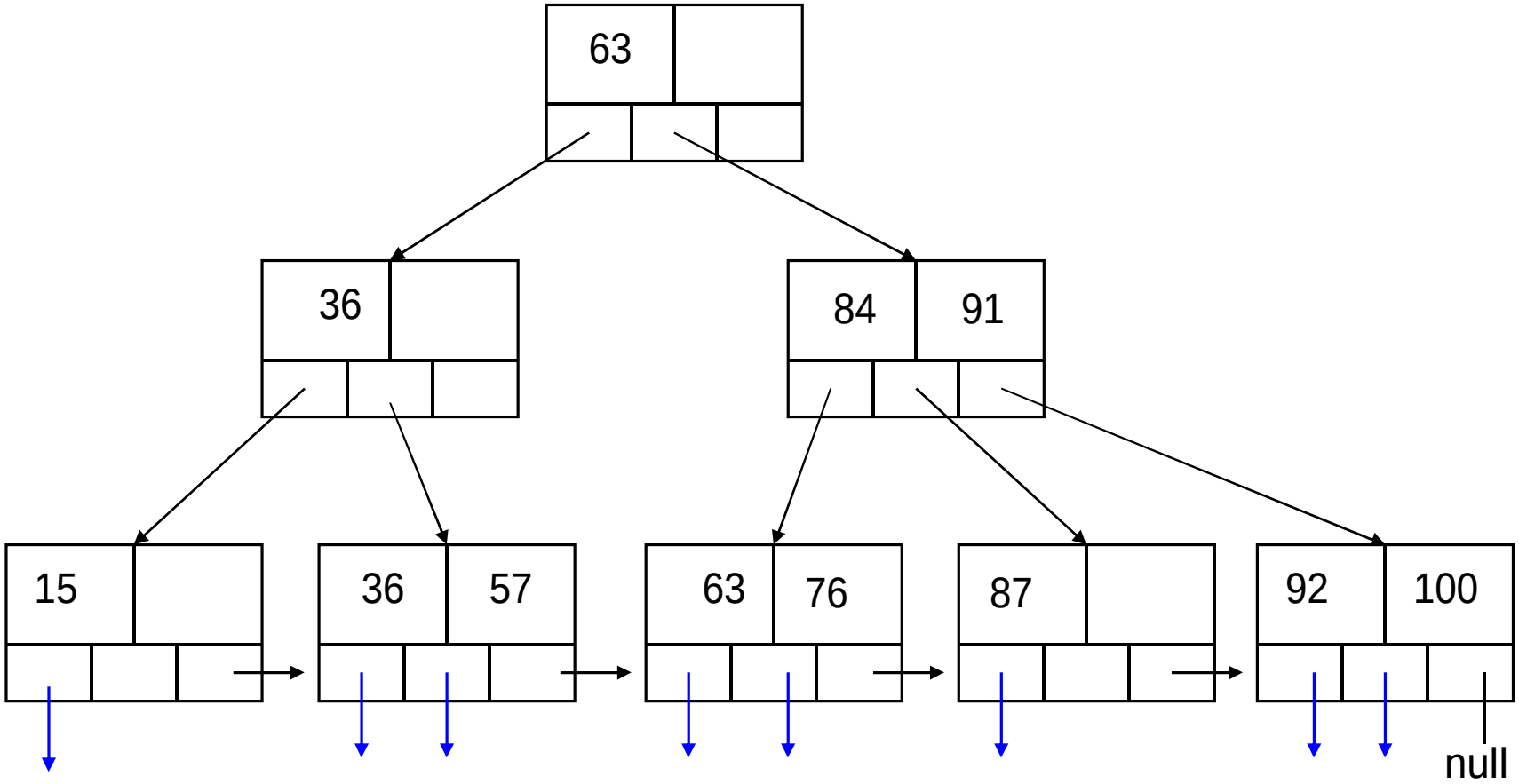




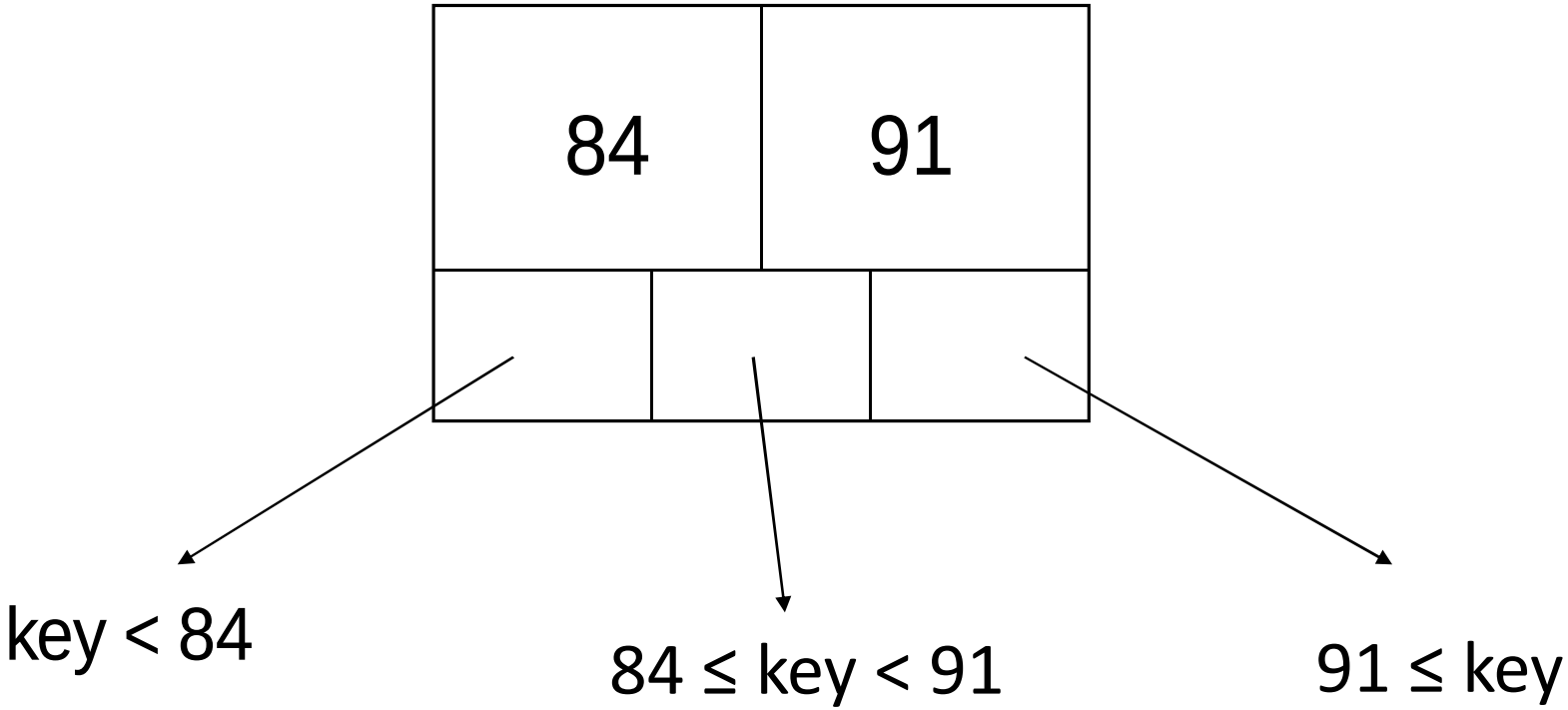
1 Random I/O prunes tree by 40

1 Random I/O prunes tree by half

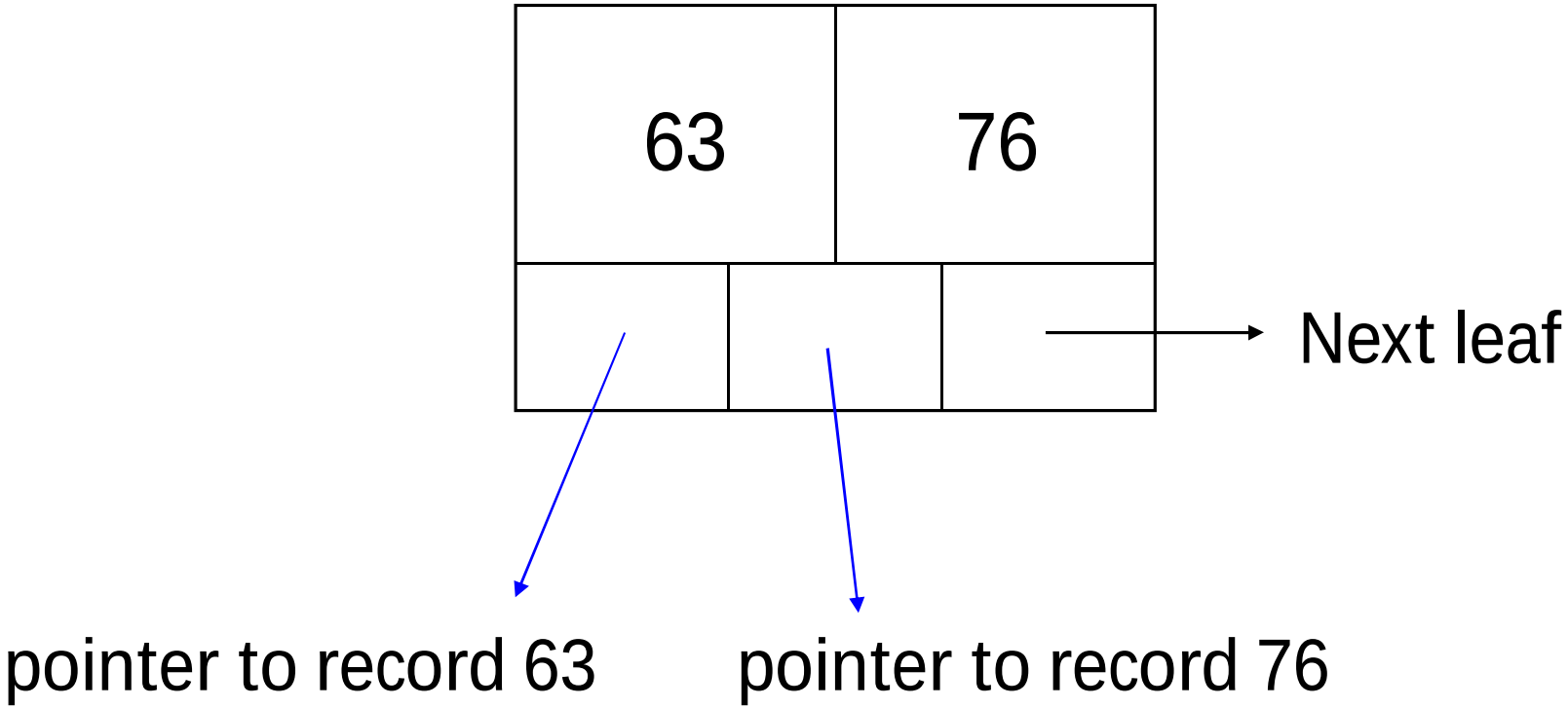
B-Tree Example



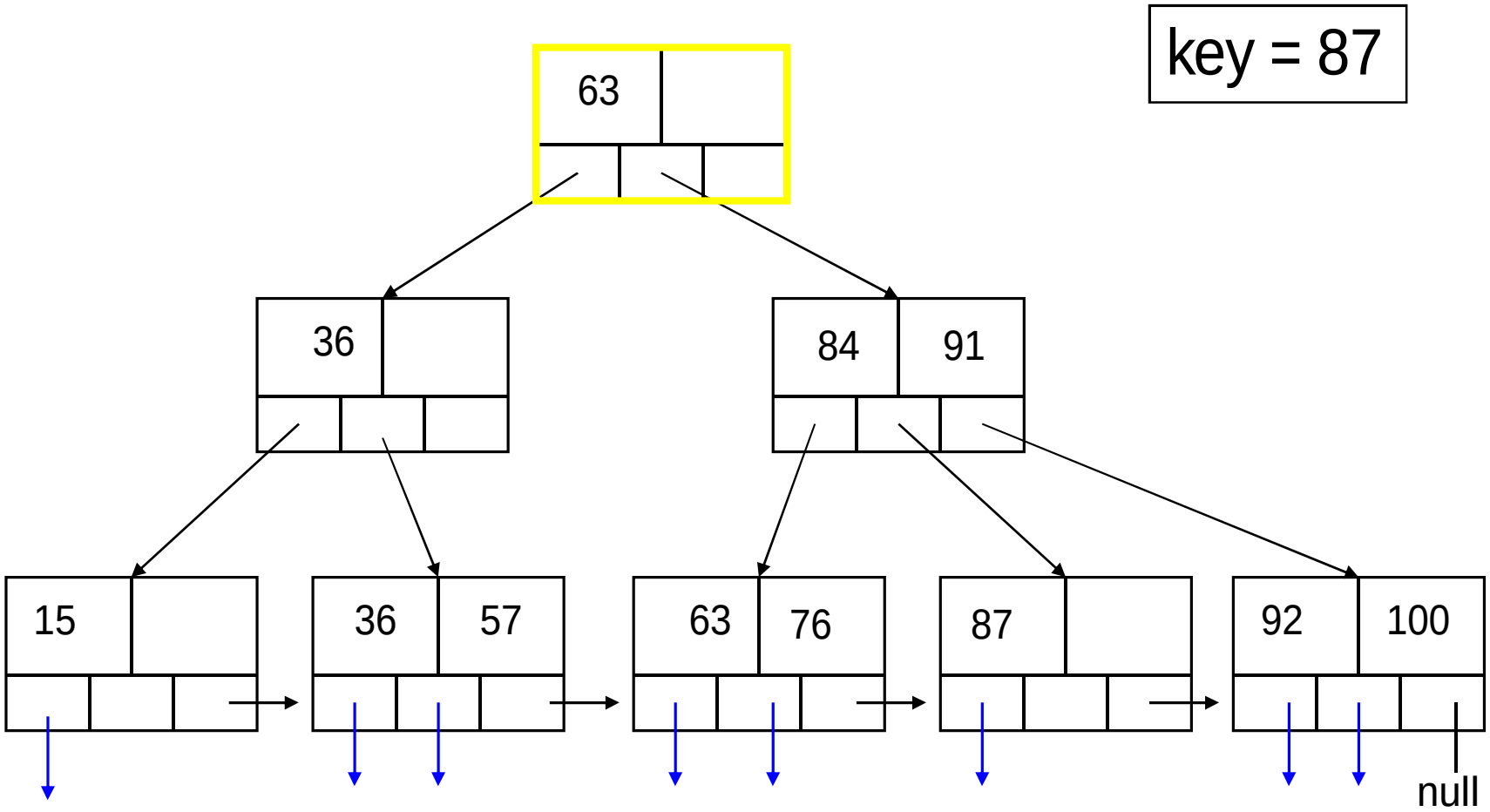
Meaning of Internal Node



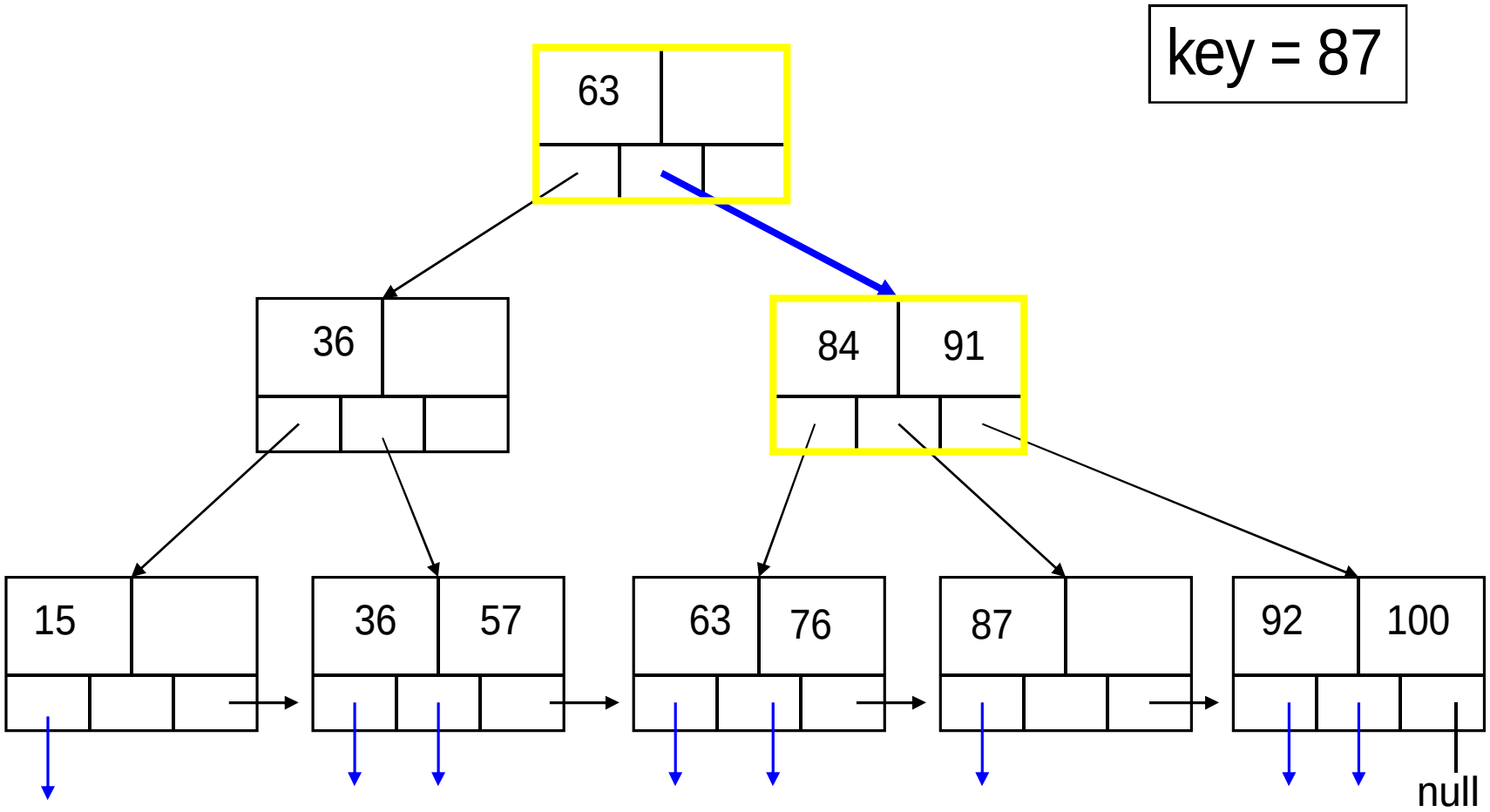
Meaning of Leaf Nodes



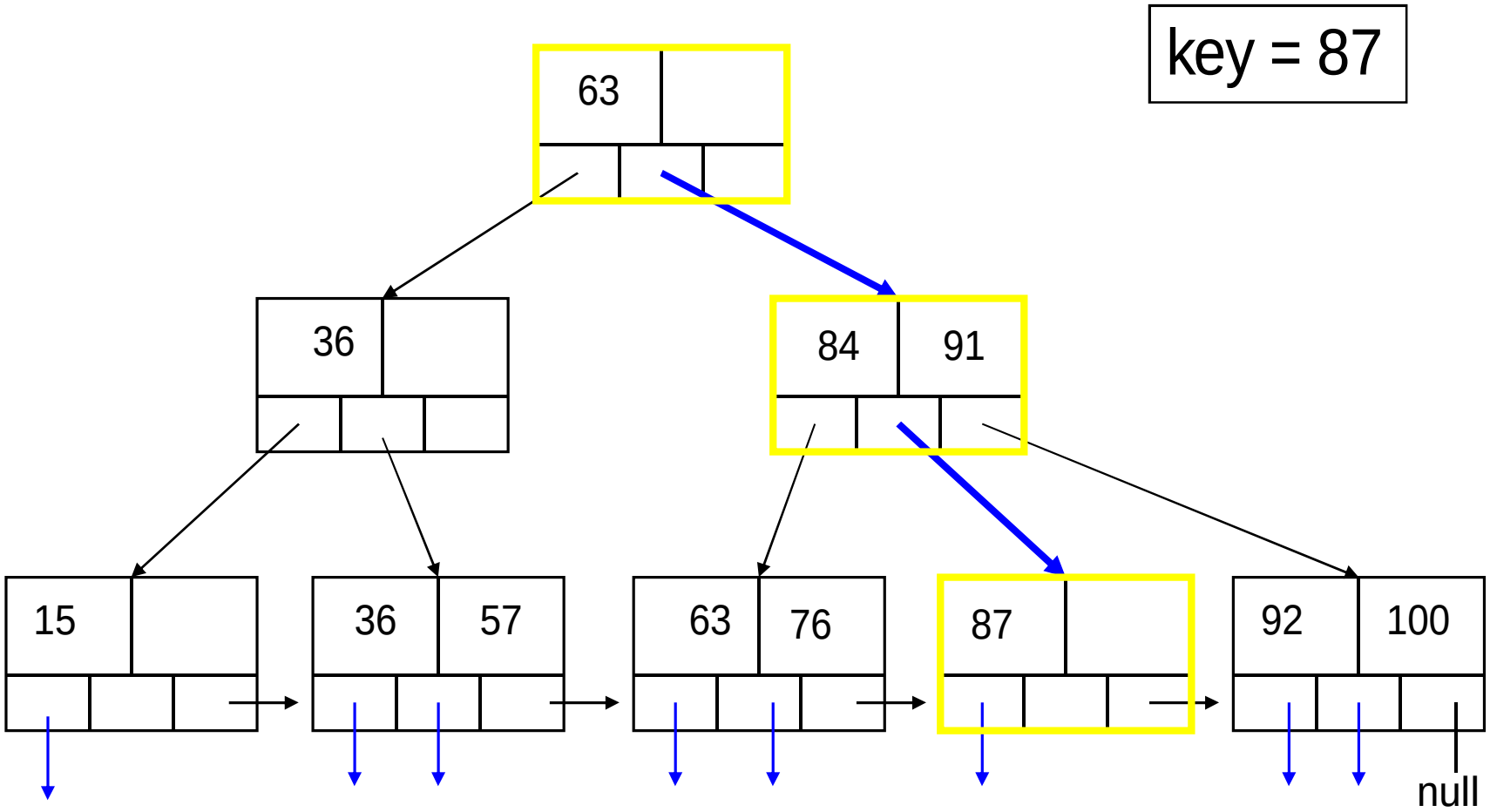
Equality Predicates



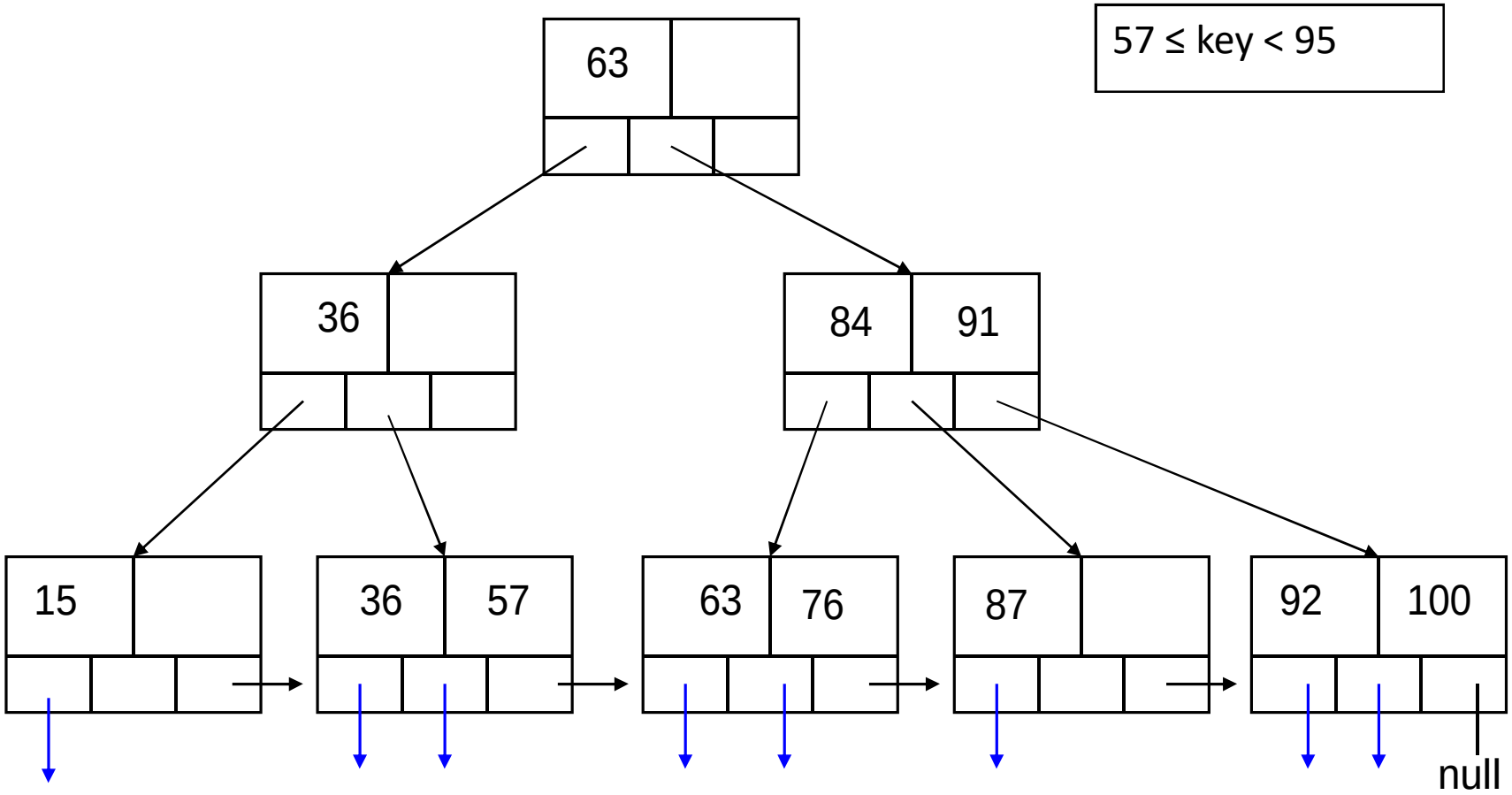
Equality Predicates



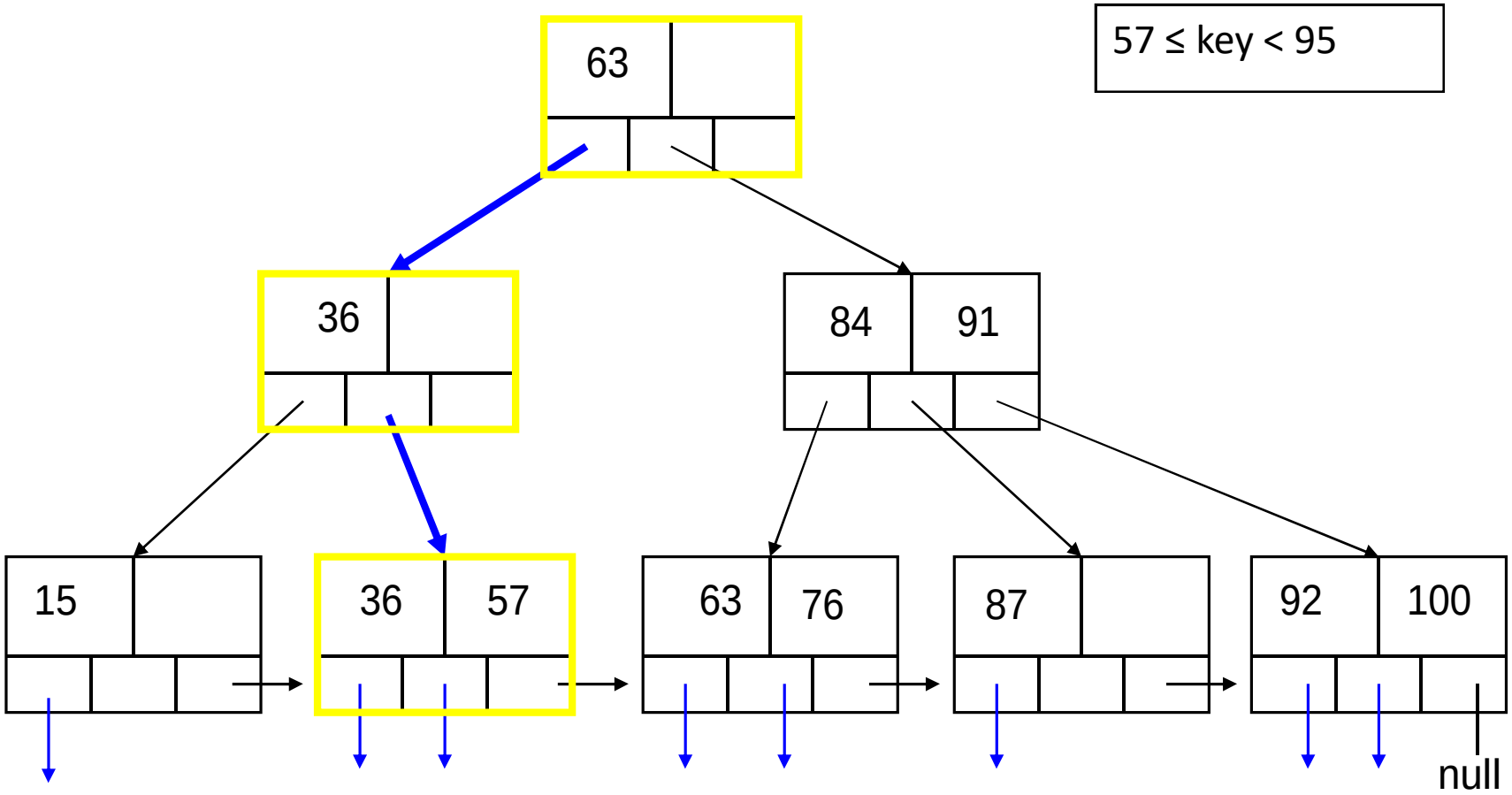
Equality Predicates



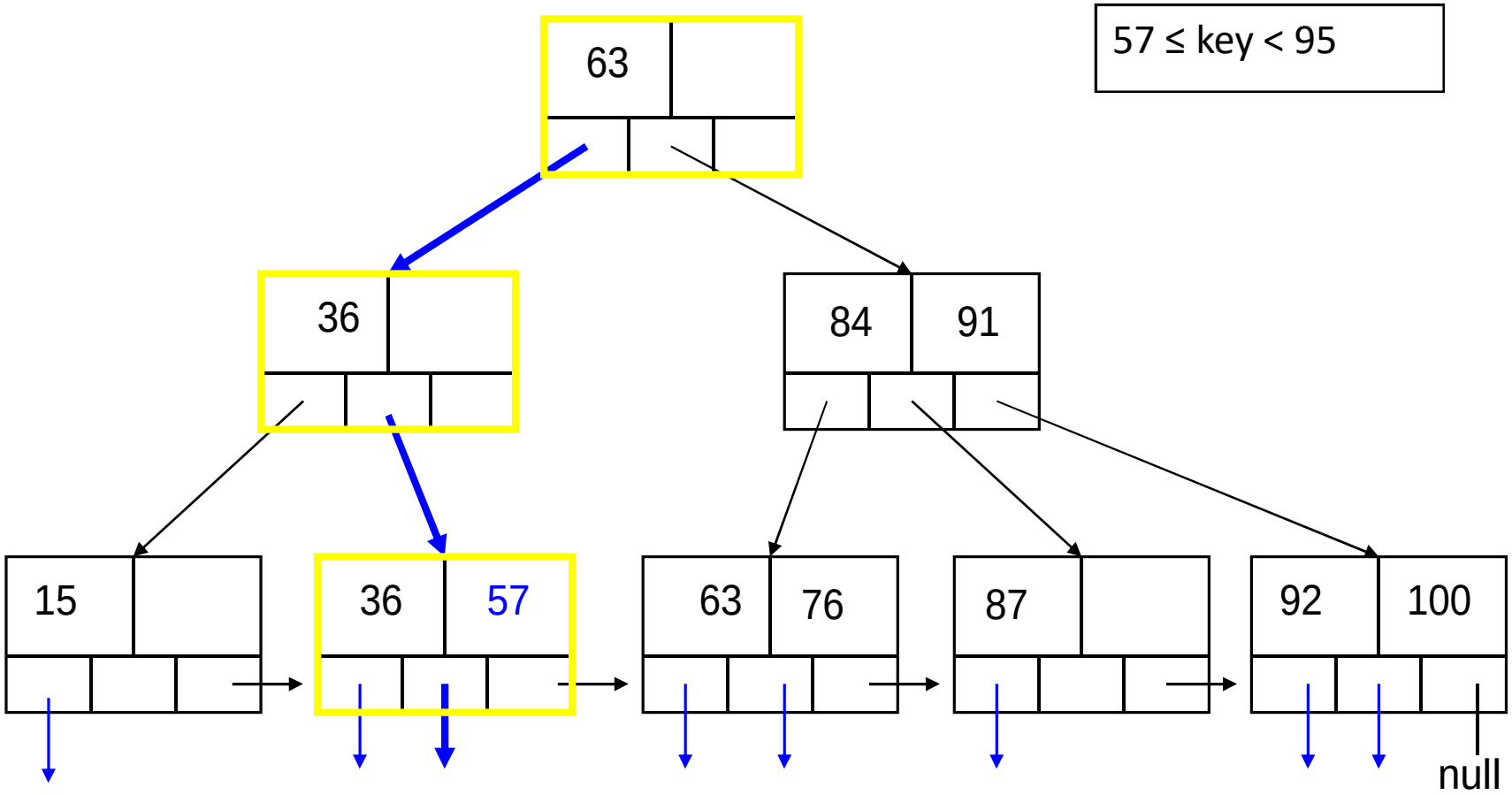
Range Predicates



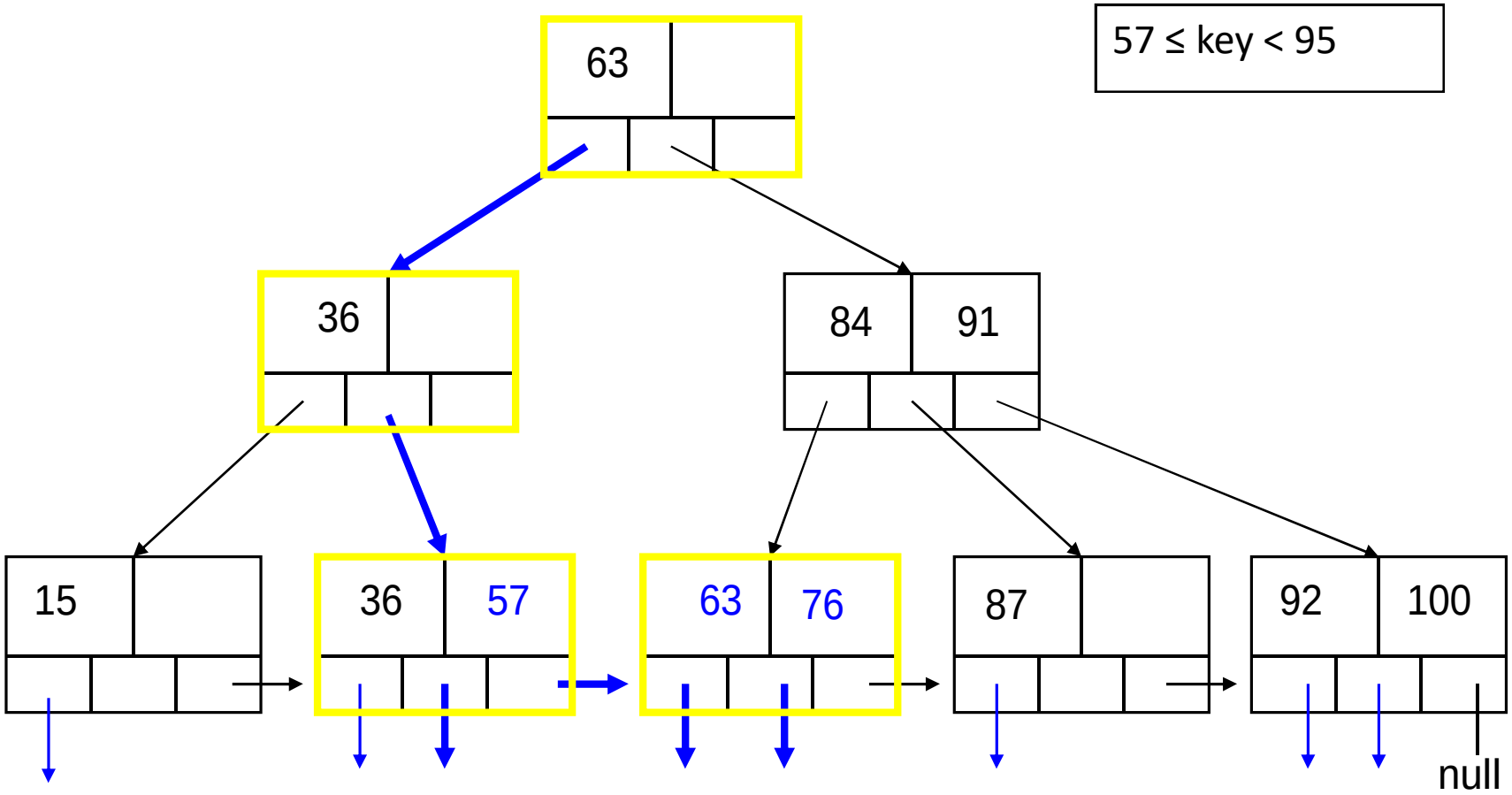
Range Predicates



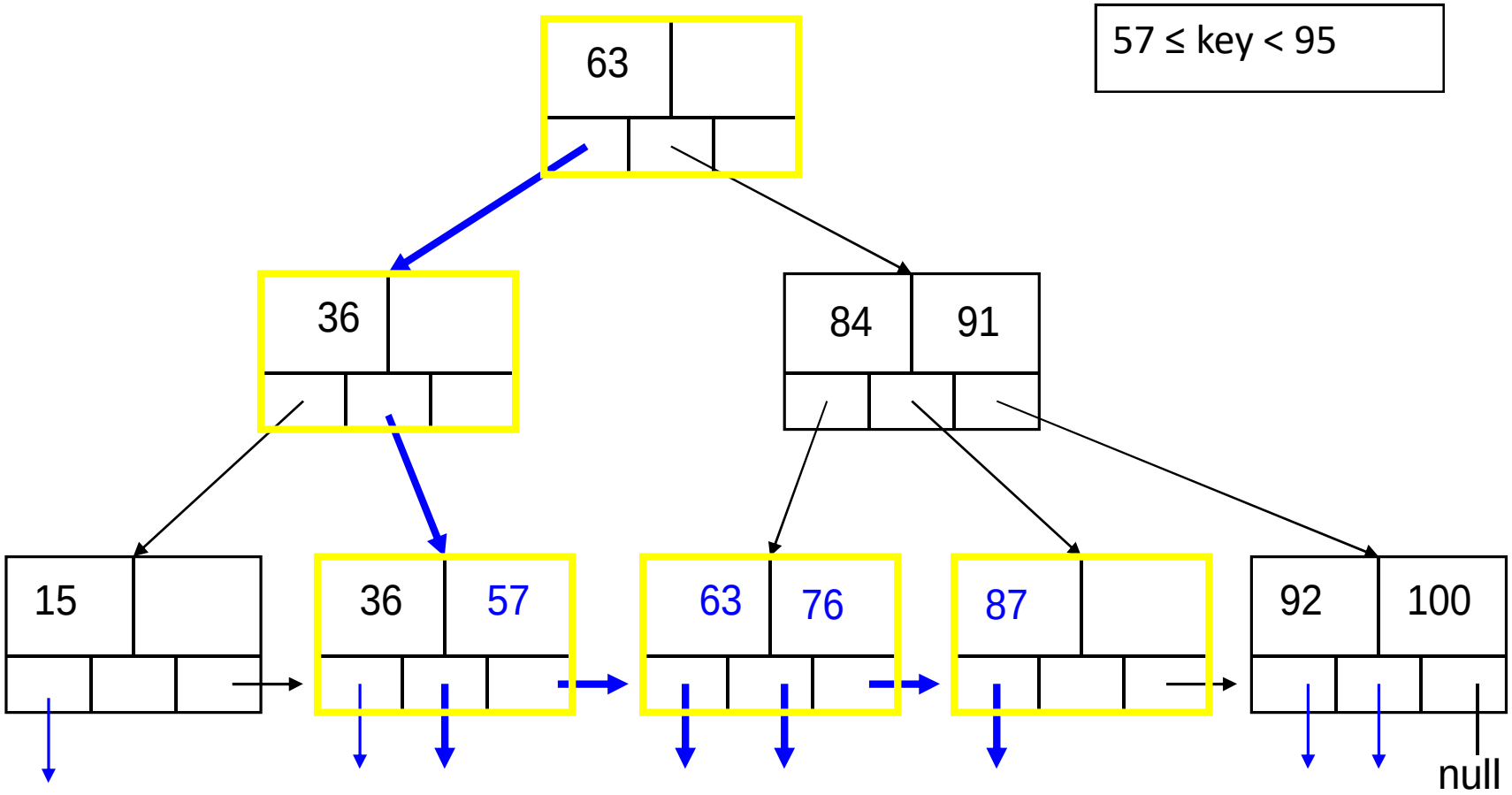
Range Predicates



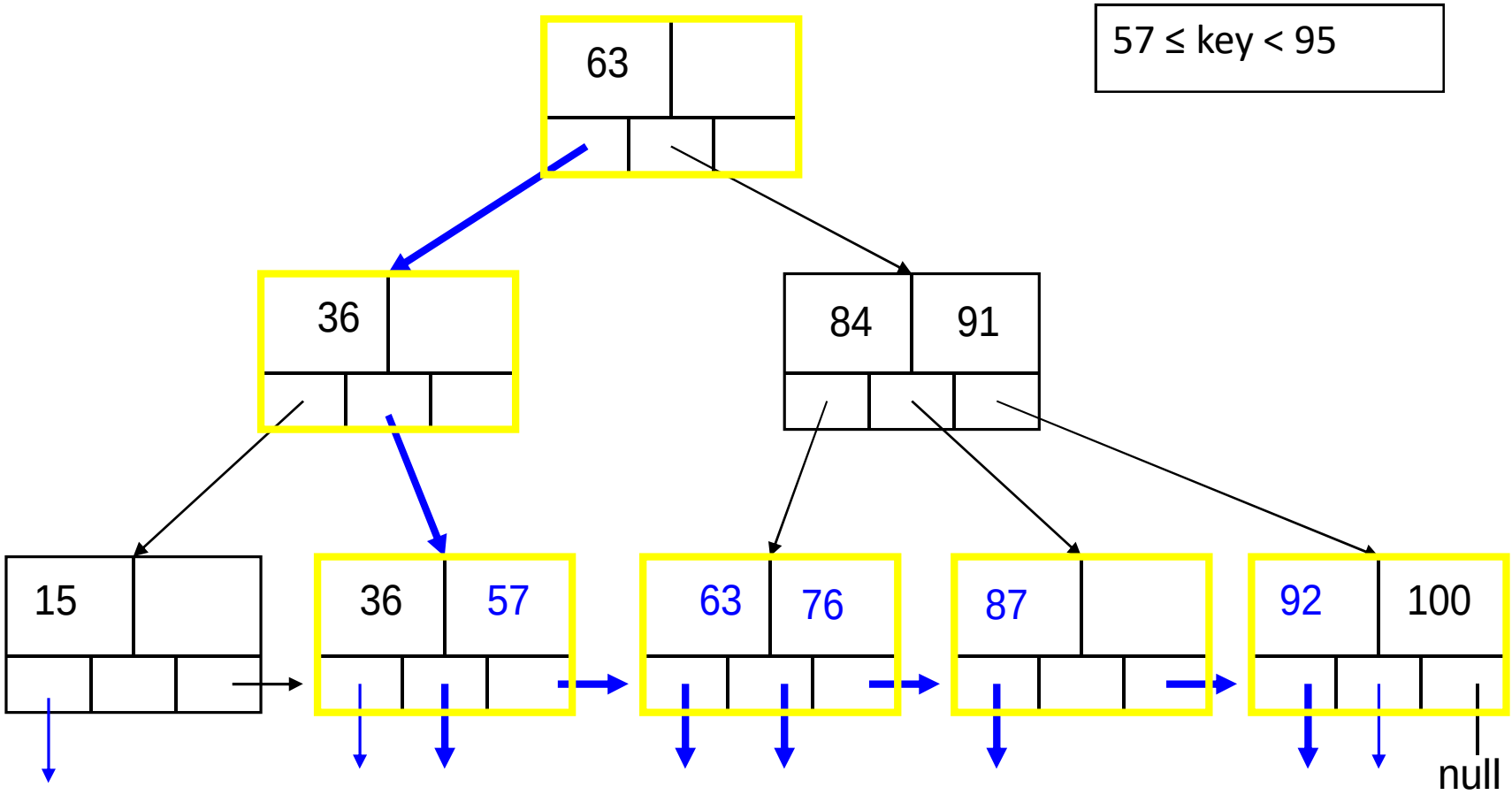
Range Predicates



Range Predicates



Range Predicates

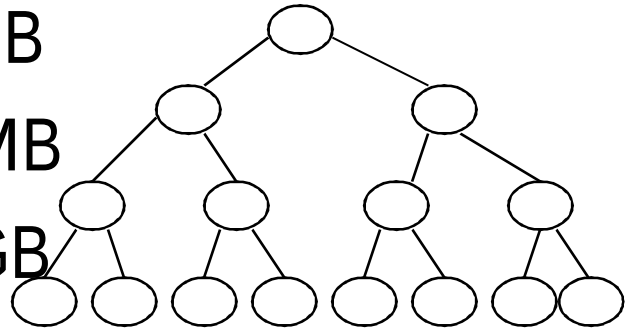


Features: B tree

- Insert/delete at $\log_F N$ cost; keep tree *height-balanced*. (F = fanout, N = # leaf pages)
- Minimum 50% occupancy (except for root). Each node contains **d** <= *m* <= 2**d** entries. The parameter **d** is called the *order* of the tree.
- Supports equality and range-searches efficiently

B+ Trees in Practice

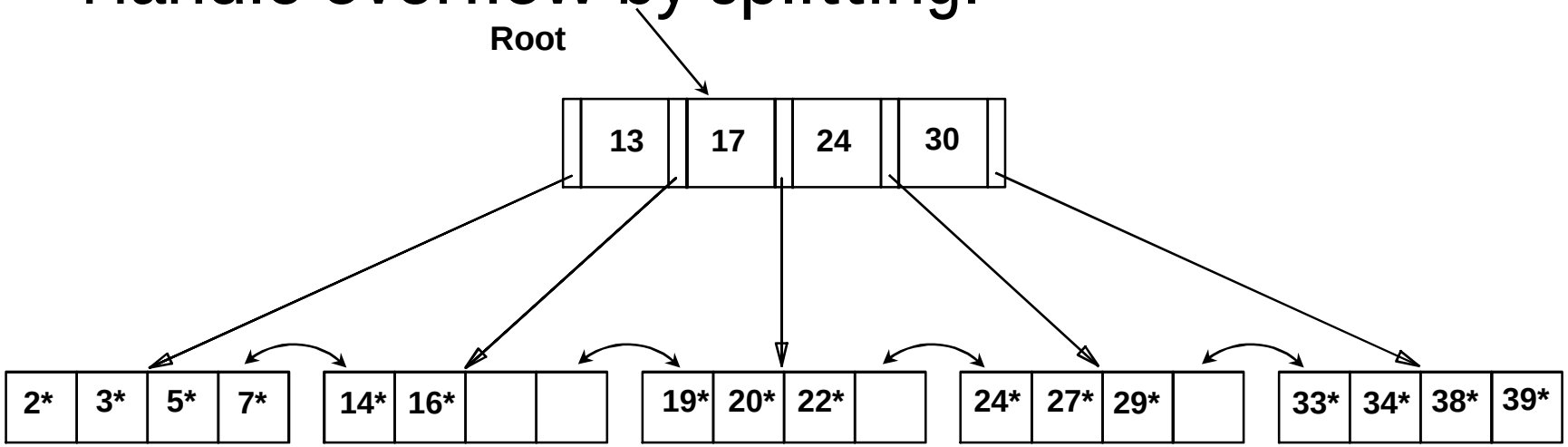
- Typical order: 100. Typical fill-factor: 67%.
  - average fanout = 133
- Can often hold top levels in buffer pool:
  - Level 1 = 1 page = 8 KB
  - Level 2 = 133 pages = 1 MB
  - Level 3 = 17,689 pages = 145 MB
  - Level 4 = 2,352,637 pages = 19 GB



❖ With 1 MB buffer, can locate one record in 19 GB (or 0.3 billion records) in two I/Os!

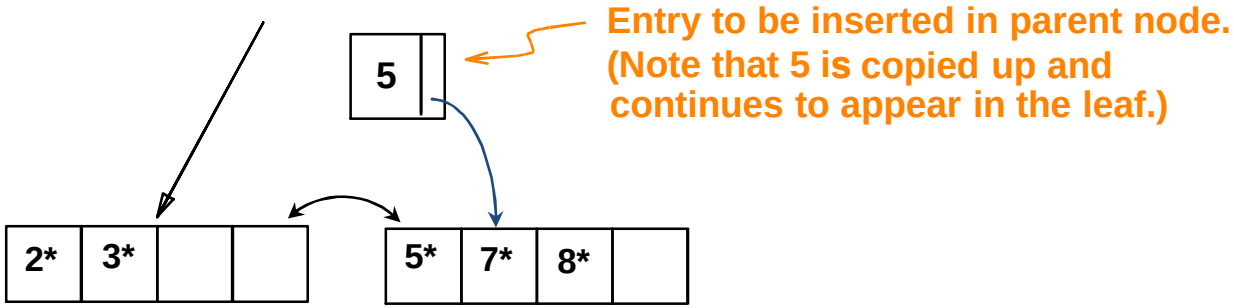
Inserting 8* into Example B+ Tree

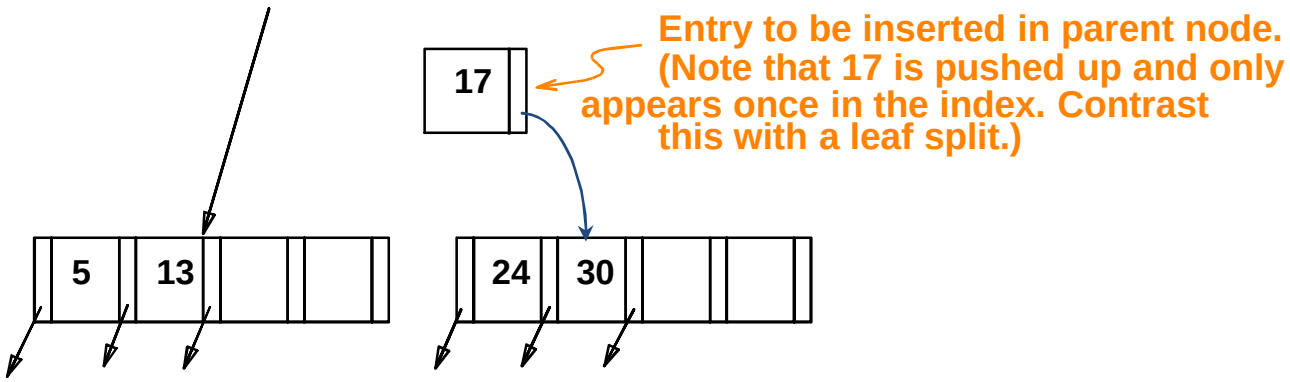
- Find leaf, in the same way as the Search algorithm.
- Handle overflow by splitting.



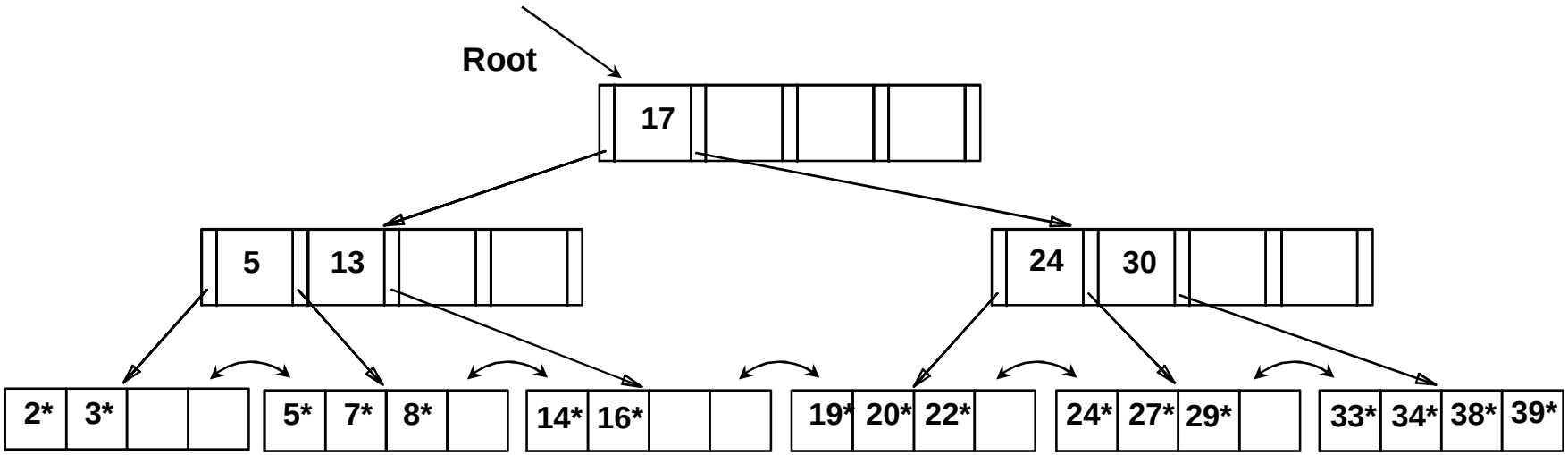
Inserting 8* into Example B+ Tree

- Observe how minimum occupancy is guaranteed in both leaf and index pg splits.
- Note difference between *copy-up* and *push-up*; be sure you understand the reasons for this.





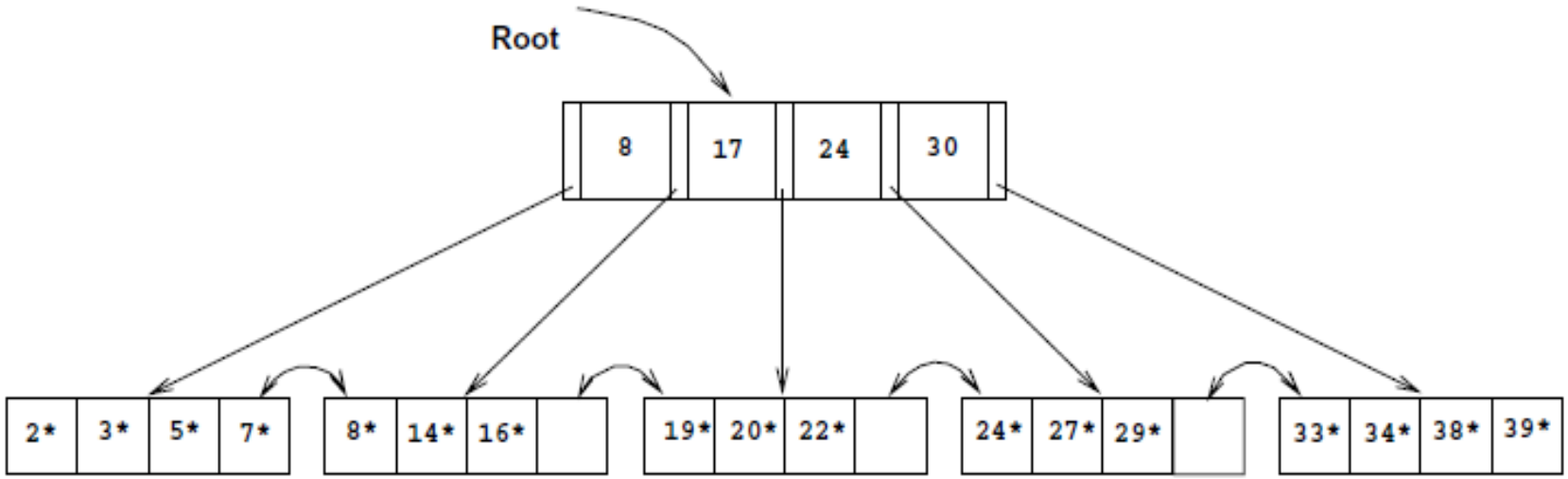
Example B+ Tree After Inserting 8*



❖ Notice that root was split, leading to increase in height.

❖ In this example, we can avoid split by re-distributing entries; however, this is usually not done in practice.

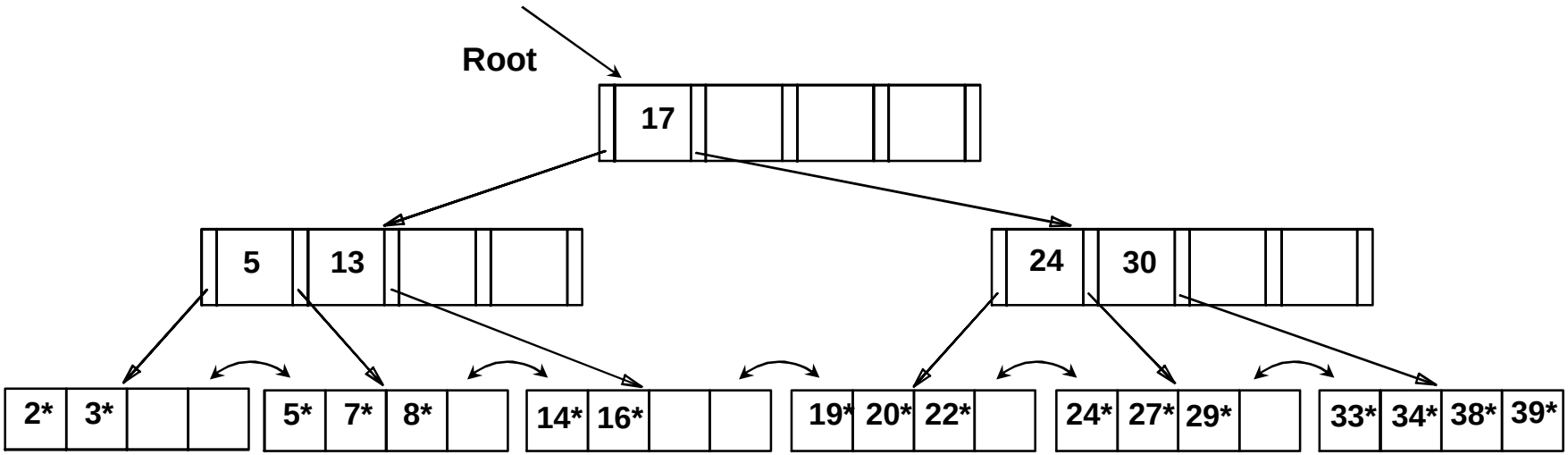
Through Sibling re-distribution



Deleting a Data Entry from a B+ Tree

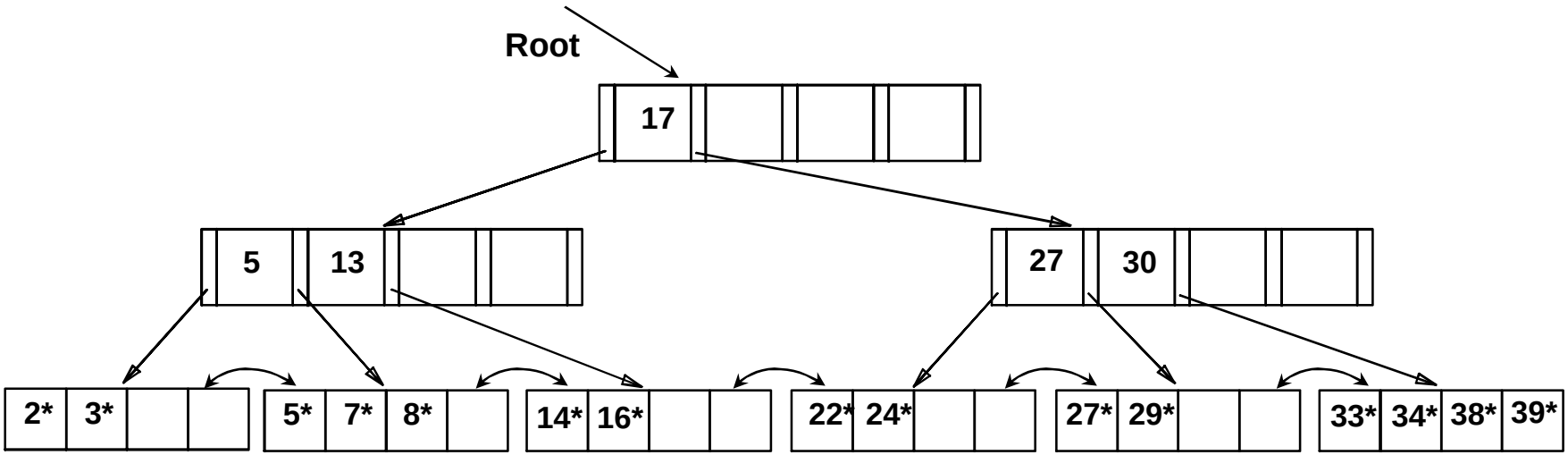
- Start at root, find leaf *L* where entry belongs.
- Remove the entry.
  - If L is at least half-full, *done!*
  - If L has only **d-1** entries,
    - Try to re-distribute, borrowing from *sibling (adjacent node with same parent as L)*.
    - If re-distribution fails, *merge* *L* and sibling.
- If merge occurred, must delete entry (pointing to *L* or sibling) from parent of *L*.
- Merge could propagate to root, decreasing height.

Deleting 19* and 20*



- Deleting 19* is easy.
- Deleting 20* is done with re-distribution.

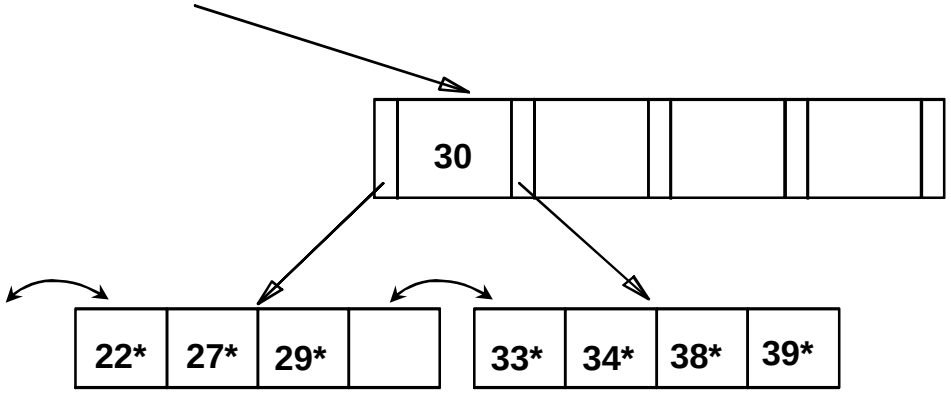
After Deleting 19* and 20*

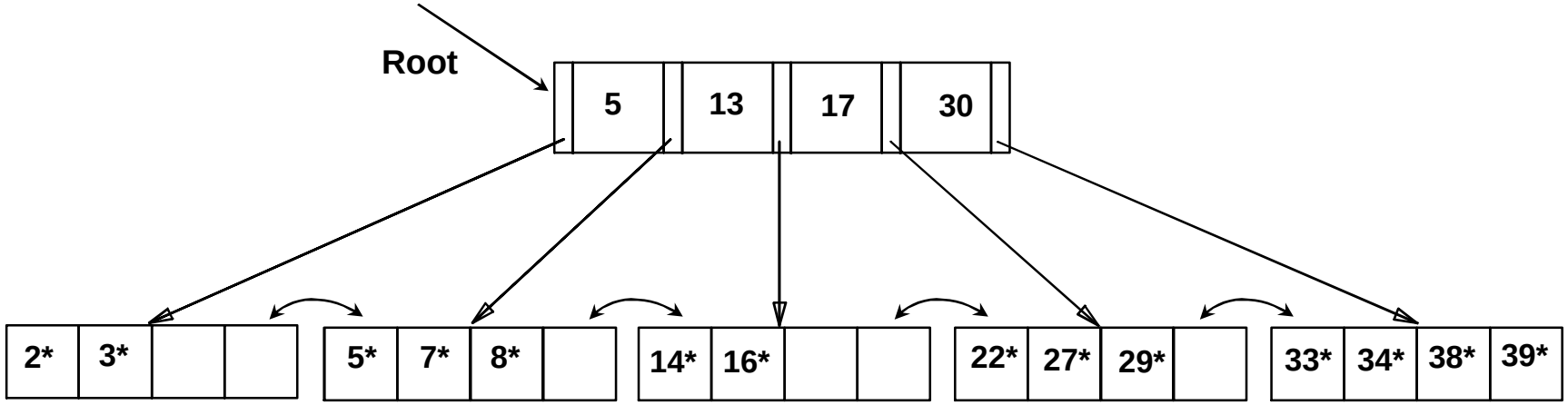


- Notice, in re-distribution, how middle key is *copied up*.
- If delete 24*...

... And Then Deleting 24*

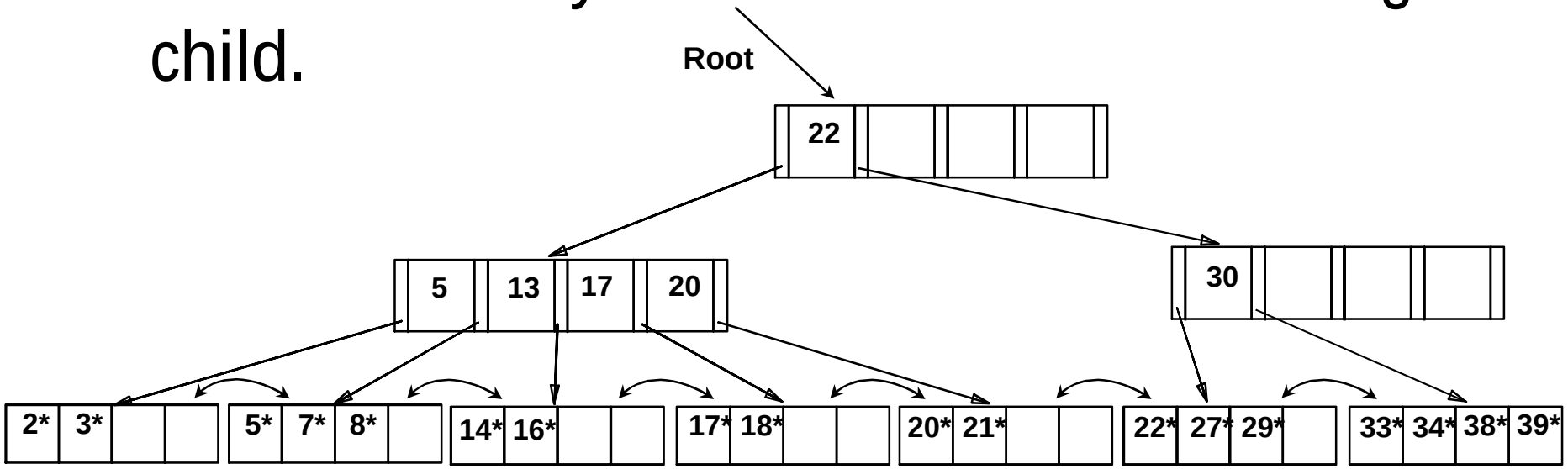
- Must merge.
- Observe `*toss*' of index entry (on right), and `*pull down*' of index entry (below).





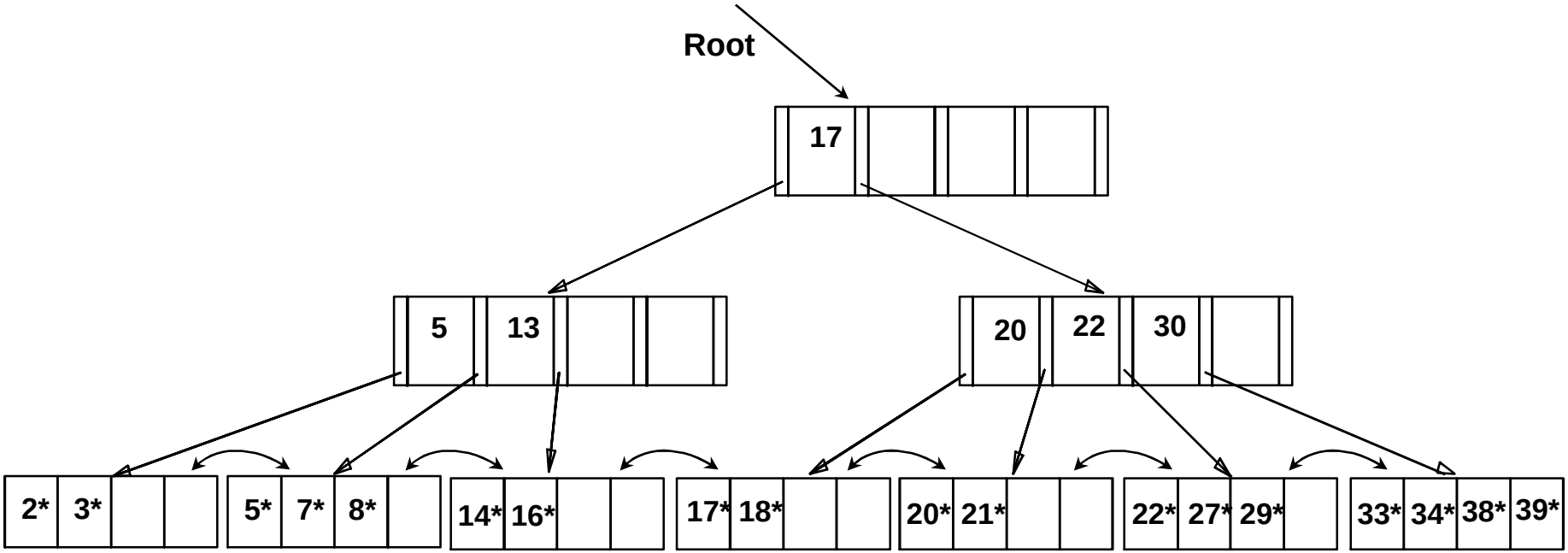
Example of Non-leaf Re-distribution

- Tree is shown below *during deletion* of 24*. (What could be a possible initial tree?)
- In contrast to previous example, can re-distribute entry from left child of root to right child.



After Re-distribution

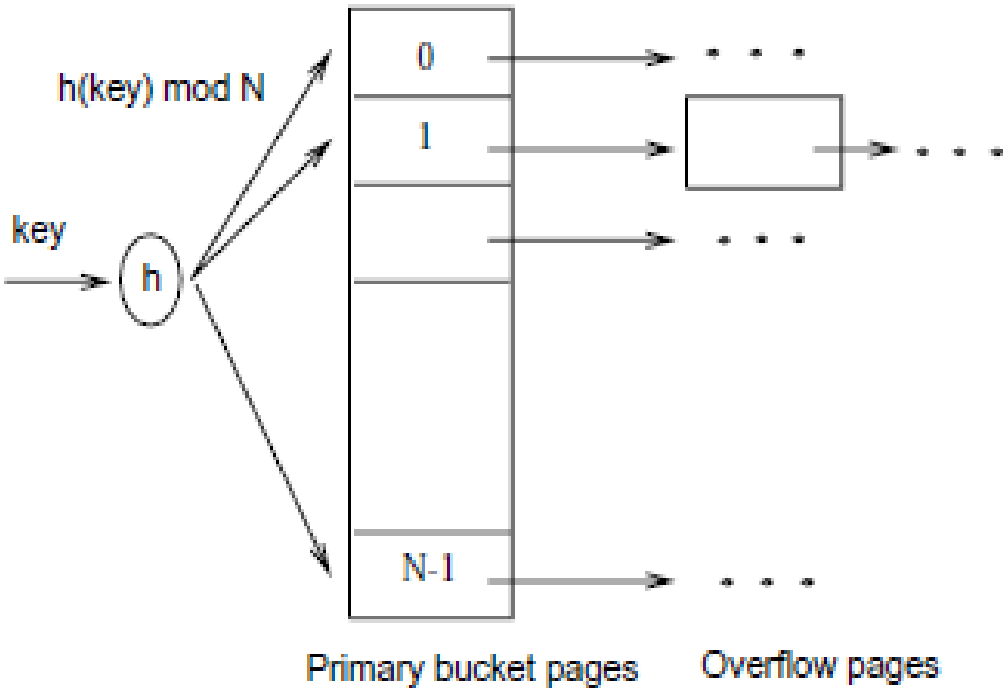
- Intuitively, entries are re-distributed by `*pushing through*' the splitting entry in the parent node.
- It suffices to re-distribute index entry with key 20; we've re-distributed 17 as well for illustration.



Features: B tree

- Insert/delete at $\log_F N$ cost; keep tree *height-balanced*. (F = fanout, N = # leaf pages)
- Minimum 50% occupancy (except for root). Each node contains **d** <= *m* <= 2**d** entries. The parameter **d** is called the *order* of the tree.
- Supports equality and range-searches efficiently

Static Hashing



A file consists of buckets o to N-1 with 1 primary page per bucket initially and the buckets hold data entries.

Static Hashing: Features

- Primary Pages are stored in successive disk pages as the number of buckets are known beforehand.
- A search ideally requires 1 disk I/Os.
- To insert & Delete it will be 2 I/Os (read & write the page)
- Cost may increase for overflow pages

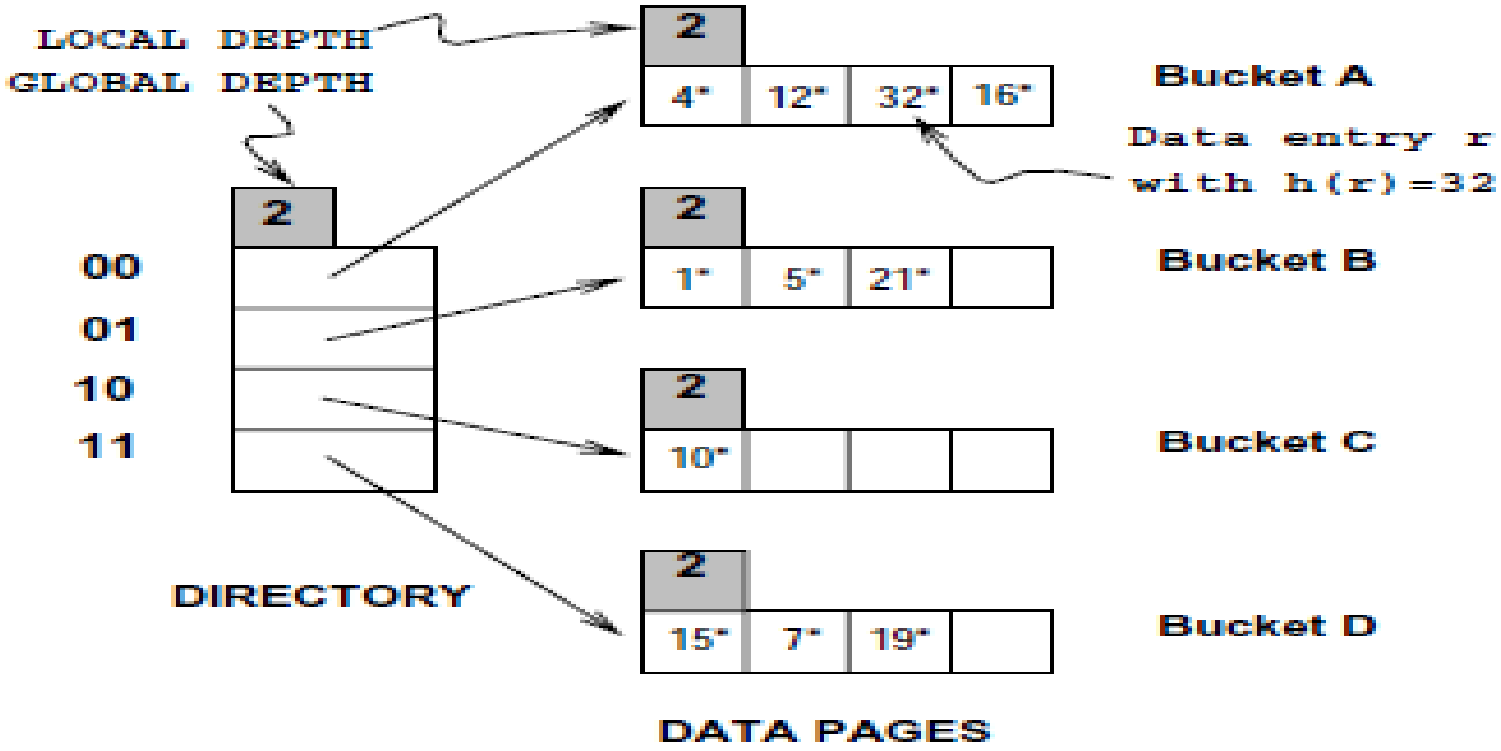
Deficiencies

- In static hashing, function *h* maps search-key values to a fixed set of *B* of bucket addresses. Databases grow or shrink with time.
  - If initial number of buckets is too small, and file grows, performance will degrade due to too much overflows.
  - If space is allocated for anticipated growth, a significant amount of space will be wasted initially (and buckets will be underfull).
  - If database shrinks, again space will be wasted.
- One solution: periodic re-organization of the file with a new hash function
  - Expensive, disrupts normal operations
- Better solution: allow the number of buckets to be modified dynamically.

Methodology: Extendebile Hashing

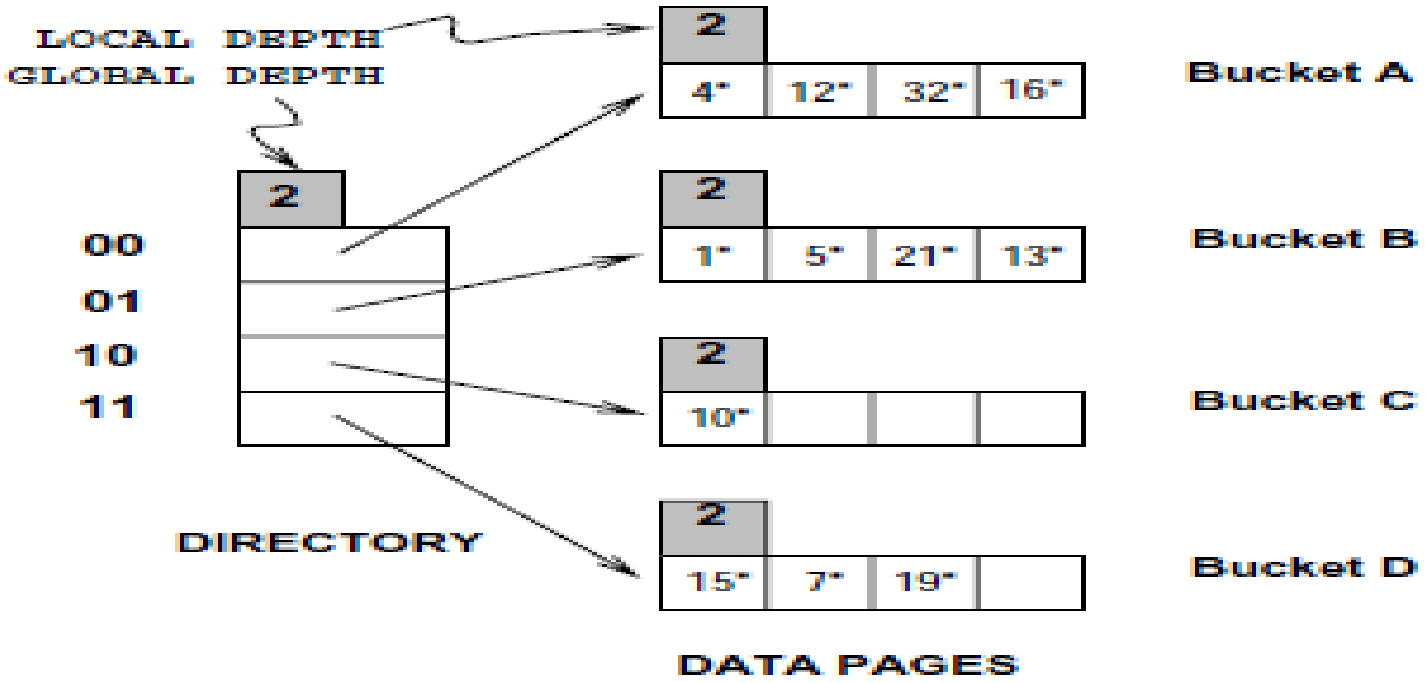
- Instead of adding overflow pages the file may be reorganized by doubling the no. of buckets and corresponding re-distribution.
- The entire file has to be read and twice as many pages have to be written.
- Alternative:
  - Use a directory of pointers to buckets
  - Double the size of the no. of buckets by doubling directory
  - Splitting only the bucket that overflowed

An example: Extendible Hashing

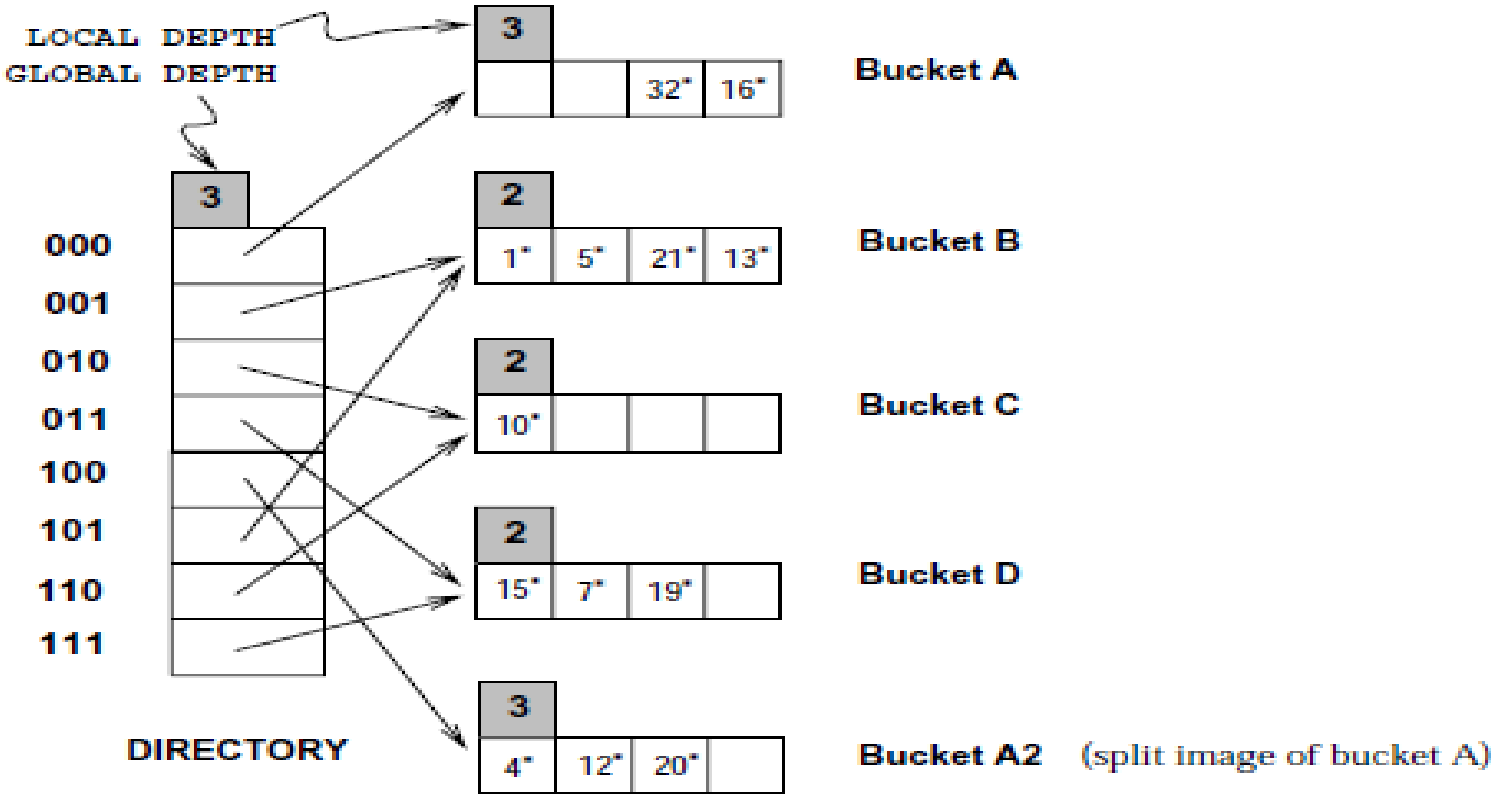


Apply the hash function on the search key and take the last 2 bits for getting desired bucket.

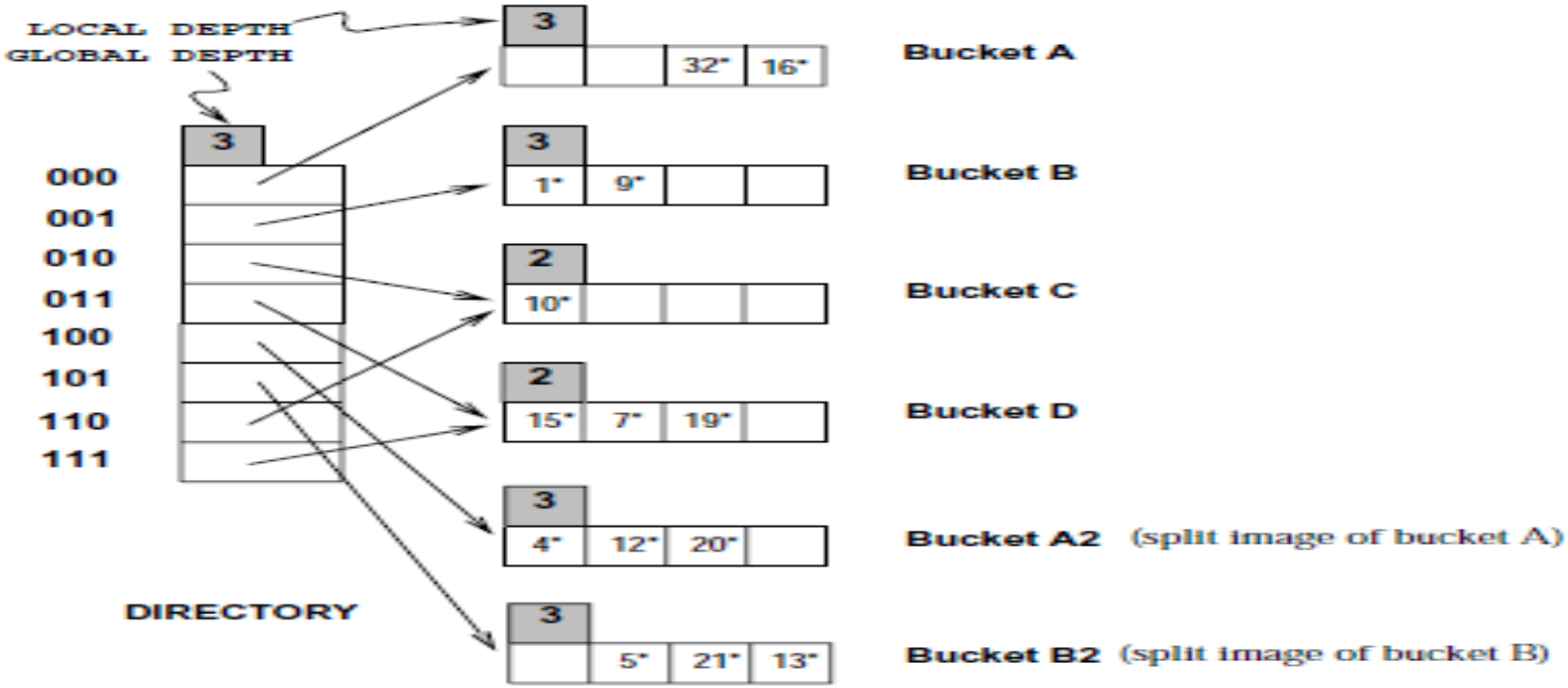
After Inserting r with h(r) = 13 [1101]



After Insertion of r with h(r) = 20 (10100)



Insertion of 9 [1001]



Always the directory doubling is not needed

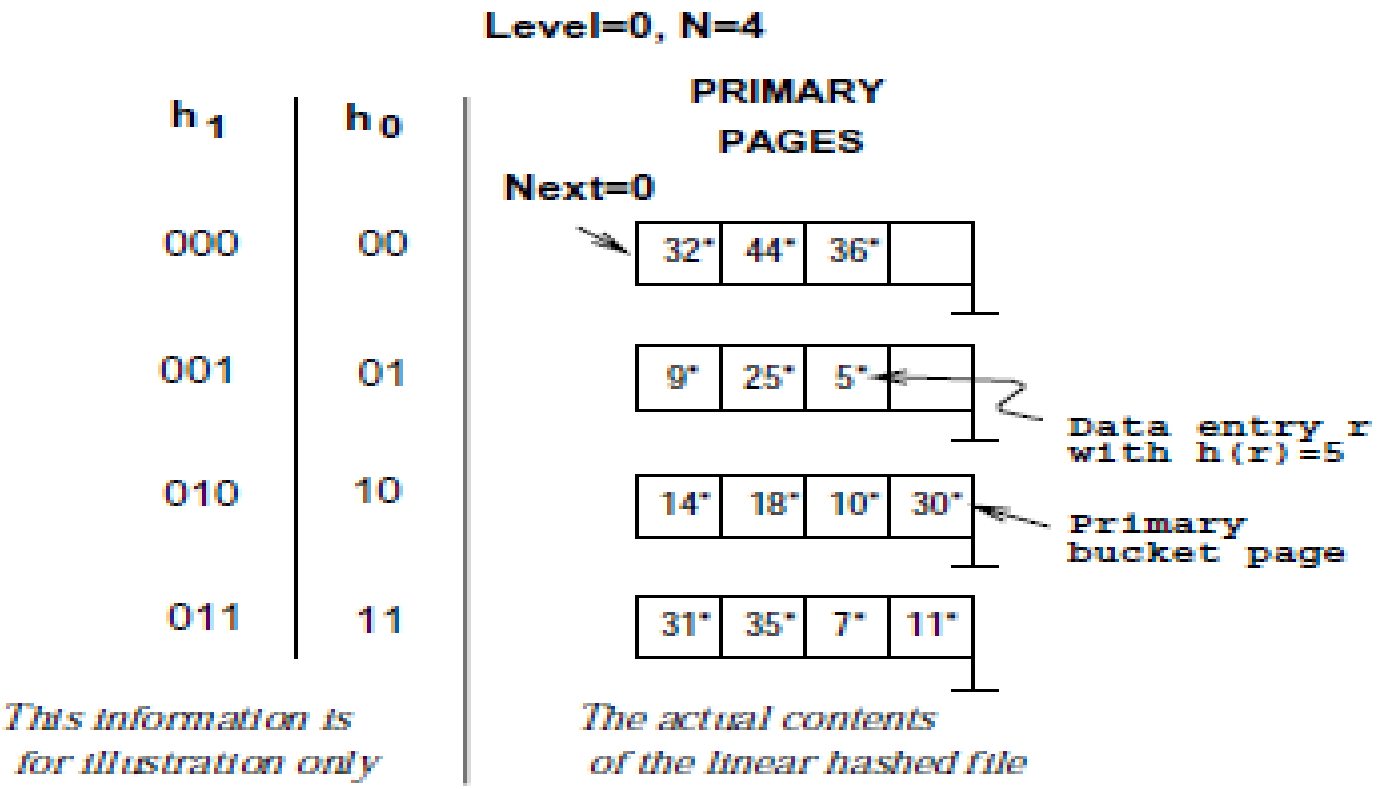
Comments: Extendeble Hashing

If the directory fits in memory, an equality selection can be answered in a single disk access, as for Static Hashing (in the absence of overflow pages), but otherwise, two disk I/Os are needed. As a typical example, a 100 MB file with 100 bytes per data entry and a page size of 4 KB contains 1,000,000 data entries and only about 25,000 elements in the directory. (Each page/bucket contains roughly 40 data entries, and we have one directory element per bucket.) Thus, although equality selections can be twice as slow as for Static Hashing files, chances are high that the directory will fit in memory and performance is the same as for Static Hashing files.

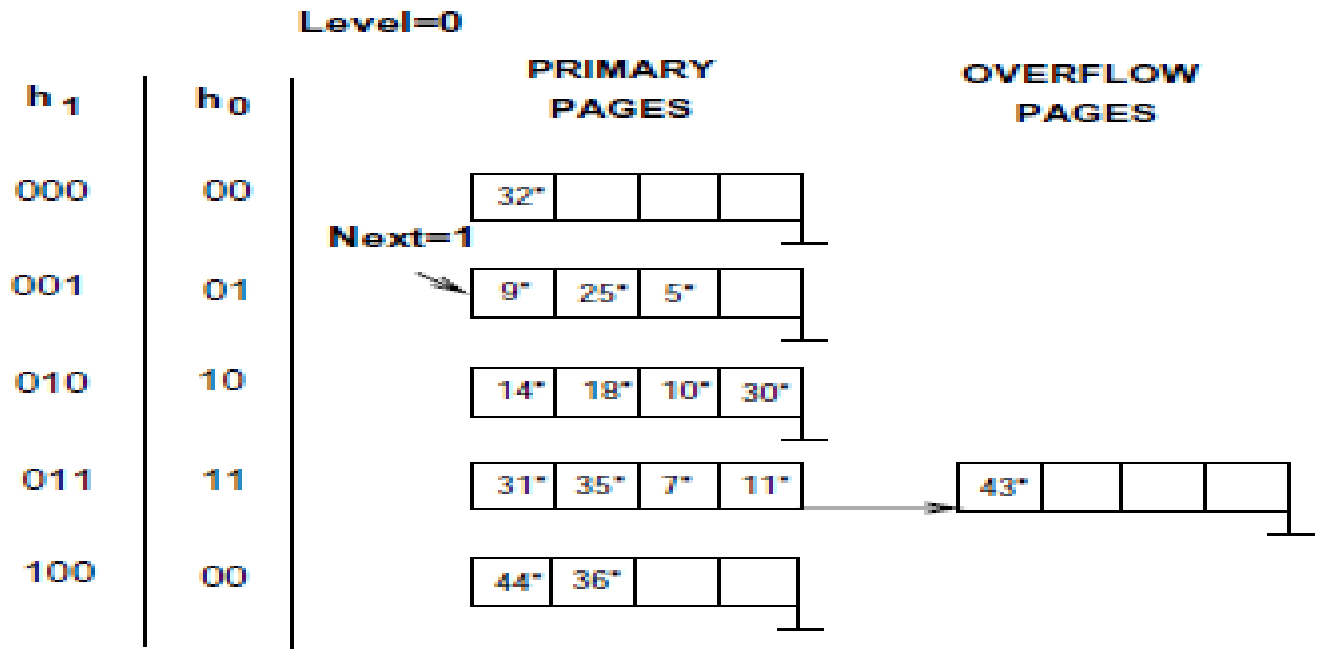
Linear Hashing

- Don’t use directory
- Utilizes a family of hash function
- H_i maps a data into m buckets where H_{i+1} maps data entry to 2M buckets

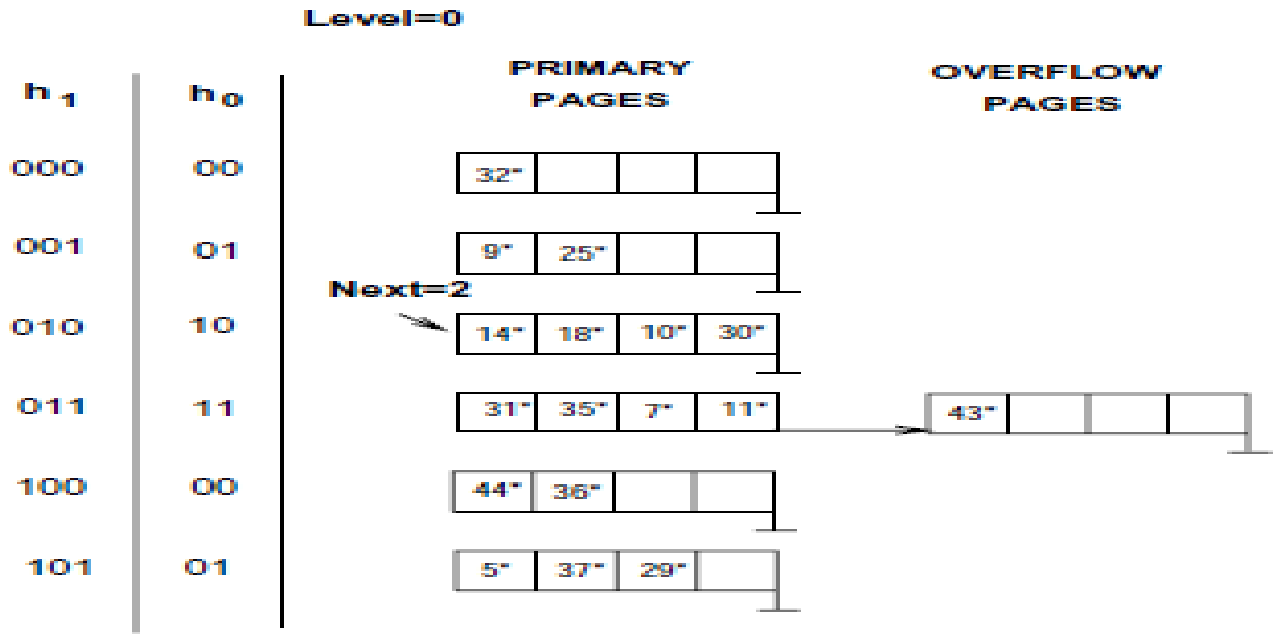
An Example: Linear Hashing



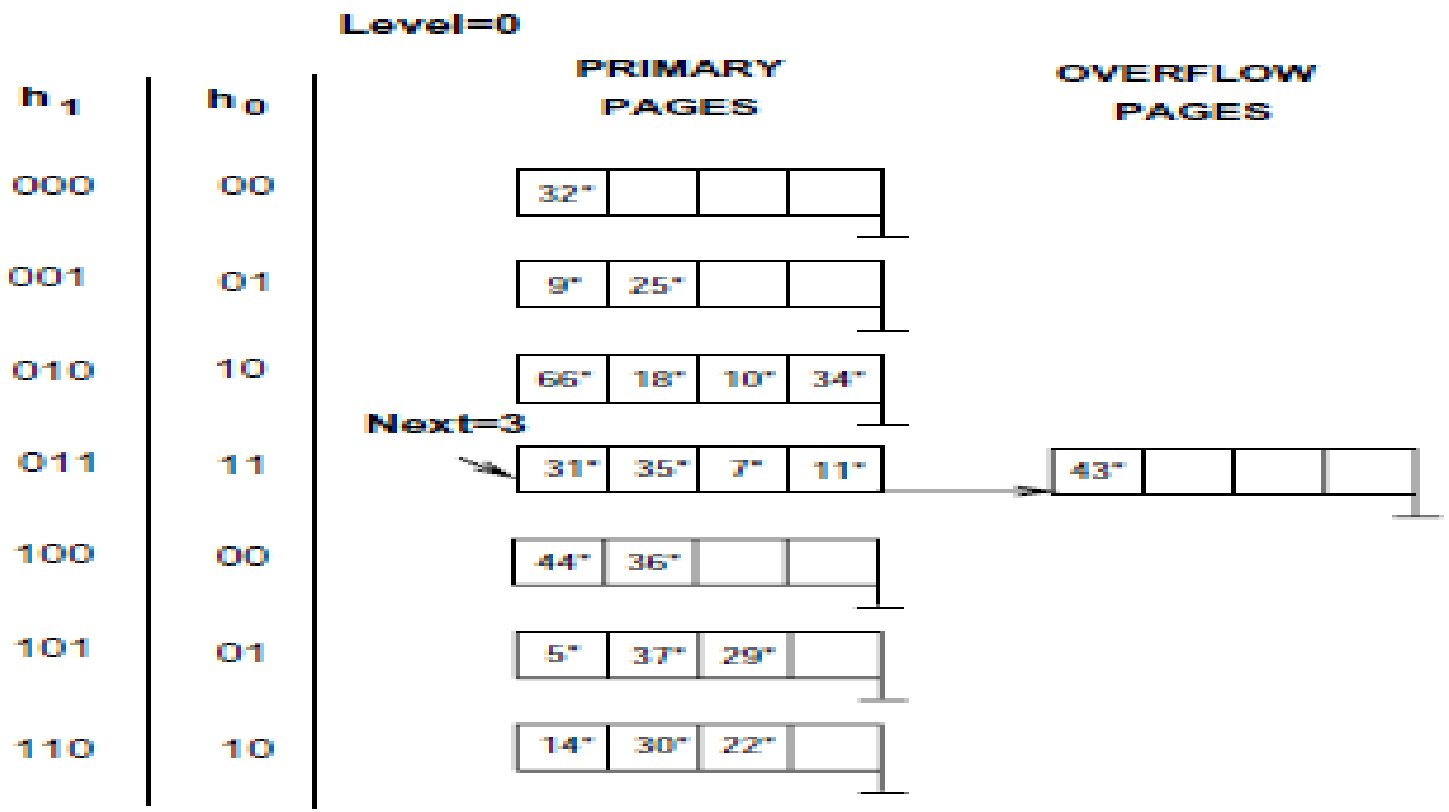
Insertion of 43 [101011]: Splitting Triggered



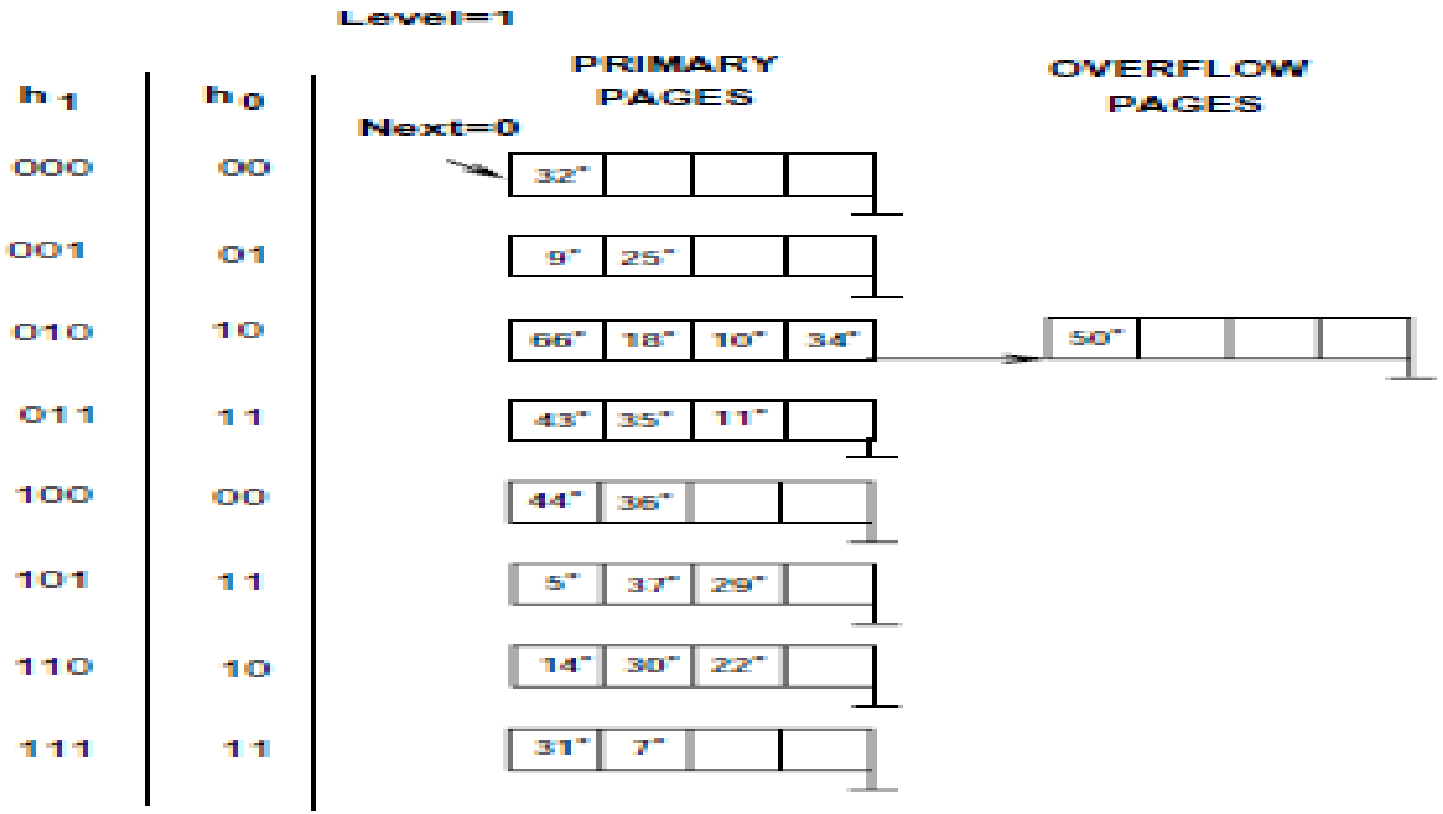
Inserting record r with h(r) = 29 [11101]



Inserting record r with h(r) = 66 (1000010), 34 (100010) & 22(10110)



Inserting record r with h(r) = 50 [110010]



Statistics in Catalog

For each base relation R:

- nTuples(R)—the number of tuples (records) in relation R (that is, its cardinality).
- bFactor(R)—the blocking factor of R (that is, the number of tuples of R that fit into one block).
- nBlocks(R)—the number of blocks required to store R. If the tuples of R are stored physically together, then:
- nBlocks(R) is equal to [nTuples(R)/bFactor(R)]
- We use [*x*] *to indicate that the result of the calculation is rounded to the smallest* integer that is greater than or equal to *x.*

Catalog: Statistics

For each attribute A of base relation R:

- nDistinct A(R)—the number of distinct values that appear for attribute A in relation R.
- minA(R), maxA(R)—the minimum and maximum possible values for the attribute A in relation R.
- SCA(R)—the *selection cardinality of attribute A in relation R. This is the average* number of tuples that satisfy an equality condition on attribute A. If we assume that the values of A are uniformly distributed in R, and that there is at least one value that satisfies the condition, then: SCA(R) = {1 if A is a key attribute of R
- [nTuples(R)/nDistinctA(R)]
- **For each multilevel index I on attribute set A:**
- • nLevelsA(I)—the number of levels in I.
- • nLfBlocksA(I)—the number of leaf blocks in I.

Cost: Selection operator

- **STRATEGIES COST**
- Linear search (unordered file, no index) [nBlocks(R)/2], for equality condition on key otherwise attribute nBlocks(R)

- Binary search (ordered file, no index) [log2 (nBlocks(R))], for equality condition on ordered attribute; more generally [log2 (nBlocks(R))] + [SCA (R)/bFactor(R)] – 1,

- Equality on hash key 1, assuming no overflow

- Equality condition on primary key nLevelsA (I) + 1
- Inequality condition on primary key nLevelsA (I) + [nBlocks(R)/2]
- Equality condition on clustering (secondary) index
  nLevelsA (I) + [SCA (R)/bFactor(R)]
- Equality condition on a nonclustering (secondary) index
  nLevelsA (I) + [SCA (R)]
- Inequality condition on a secondary B plus tree index
  nLevelsA (I) + [nLfBlocksA (I)/2 + nTuples(R)]

Thank you